

基于稀疏二阶锥规划的火星动力下降轨迹优化

LShang

2026 年 7 月 13 日

摘要

本文以火星动力下降段轨迹优化为工程背景，围绕序列锥规划（Sequential Convex Programming, SCP）框架下的二阶锥规划（Second-Order Cone Programming, SOCP）问题展开研究。针对动力下降段中推力幅值禁带约束、下滑角约束和终端状态约束等非凸因素，采用无损凸化技术将其等价转换为凸约束，建立标准 SOCP 问题的完整数学形式。为实现嵌入式环境下的高效求解，本文手工构造了完整的稀疏矩阵数据结构，其中等式约束 Jacobian 矩阵包含 733 个非零元，不等式约束 Jacobian 矩阵包含 403 个非零元，完全消除对通用建模工具（CVXPY、YALMIP 等）的运行时依赖。在此基础上构建了涵盖六种求解器形态的交叉验证体系：通用内点求解器 ECOS 与 Clarabel、非线性规划求解器 IPOPT、嵌入式求解器 acados，以及两种自主实现的 C 语言求解器。六套求解器在相同 SOCP 问题上独立计算，最终燃耗结果均收敛于 400.7 kg，与文献中的黄金基准严格一致，验证了 SOCP 建模的正确性与求解器的数值可靠性。此外，针对嵌入式部署需求，本文在 x86 平台完成了求解器的内存占用与计算延迟评估，为后续星载实现提供了基础数据支撑。

1 引言

1.1 研究背景

火星探测是人类深空探索的重要里程碑。迄今为止，全球仅有美国和中国成功实现了火星表面的软着陆，而苏联、欧洲和日本的尝试均以失败告终。着陆任务的成功实施依赖于动力下降段（Powered Descent Phase）的精确轨迹规划与控制。该阶段通常始于约 28 km 高度、速度约 100 450 m/s，终止于着陆器触地时刻的六自由度状态归零 [1]。动力下降段持续时间短（约 60 90 s）、燃料消耗大，是整个 EDL（Entry, Descent, Landing）过程中制导精度要求最高的阶段——在此阶段，着陆器已无法通过降落伞或大气减速进一步调整，仅依赖反推发动机实现从超音速到软触地的完整机动。任何轨迹规划层面的偏差都可能导致着陆位置大幅偏离目标甚至任务失败，因此对制导算法的实时性、鲁棒性和燃料最优性提出了极高的要求。

美国国家航空航天局（NASA）在火星着陆领域取得了开创性的成就。2012 年火星科学实验室（Mars Science Laboratory, MSL）任务首次采用天“空起重机”（Sky Crane）方案：在动力下降段末段将“好奇号”（Curiosity）火星车以三根缆绳悬吊释放，实现了亚米级着陆精度 [2]，标志着火星着陆由千米级椭圆进入百米级精度时代 [2]。2021 年，火星 2020（Mars 2020）任务通过“毅力号”（Perseverance）火星车引入了航程触发（Range Trigger）和地形相对导航（TRN）技术，进一步将着陆椭圆半径缩小至百米量级 [3]，同时首次在动力下降段使用多光谱相机进行实时地形匹配避障 [3]。此外，火星 2020 装备的 MEDLI2（Mars Entry, Descent, and Landing

Instrumentation 2) 传感器套件采集了进入段和下降段的高保真气动热数据, 为后续任务的制导模型校核提供了宝贵依据。中国国家航天局 (CNSA) 于 2021 年 5 月成功实现了“天问一号” (Tianwen-1) 任务的火星软着陆, 使中国成为世界上第二个独立实现火星着陆的国家 [4]。天问一号着陆器采用伞降-动力下降复合方案, 其动力下降段通过六台节流式液体火箭发动机实现推力连续调节, 最终确保了“祝融号” (Zhurong) 火星车在乌托邦平原的安全着陆。

上述任务中, 动力下降段的轨迹优化面临三大核心挑战。其一, 系统动力学呈现高度非线性, 且受火星重力场、大气阻力等多物理场耦合影响, 运动方程在惯性系中呈现强耦合的六自由度特征。其二, 推力器存在严格的物理约束——发动机推力幅值受限于零和最大推力之间的非凸禁带 ($T_{\min}$ 与 $T_{\max}$ 之间的不可调区间), 推力方向受摆动角限制, 这些约束在数学上构成非凸集合, 无法直接应用凸优化理论 [5]。其三, 着陆场景要求星载计算机在数秒至数十秒内完成轨迹重规划, 若遭遇阵风或地形突变等意外情况, 甚至需要在秒级以内给出新轨迹, 实时性要求极为苛刻。

1.2 研究现状

动力下降轨迹优化的求解方法可归纳为间接法、直接法和凸优化方法三大流派, 其发展历程反映了理论严谨性与工程实用性之间的持续博弈。

间接法基于庞特里亚金极大值原理 (Pontryagin's Maximum Principle), 通过引入协态变量将原最优控制问题转化为两点边值问题 (Two-Point Boundary Value Problem, TPBVP)。该方法的最优性条件严格, 最优解精确满足一阶必要条件, 适合对求解精度要求极高的离线分析场景。然而, 间接法面临三大固有困难: 一是 TPBVP 的求解对协态变量初值猜测极为敏感, 二阶震荡导致迭代难以收敛; 二是推力约束中的开关切换结构 (Bang-Bang 控制) 使 Hamiltonian 函数不可微, 加剧了数值求解的振荡; 三是间接法难以处理路径状态不等式约束 (如地形障碍规避和禁飞区约束), 限制了其在实际着陆场景中的推广 [6]。

直接法通过将连续时间最优控制问题离散化为有限维非线性规划 (Nonlinear Programming, NLP) 问题, 采用伪谱法 (Pseudospectral Method) 或配点法 (Collocation Method) 进行求解。伪谱法在 Legendre-Gauss-Lobatto 或 Chebyshev 节点上对状态变量和控制变量进行全局拉格朗日多项式逼近, 具有谱收敛率, 在节点数较少的条件下即可获得高精度解 [7]。配点法则将轨迹在时间上分段, 每段采用低阶多项式的 Hermite-Simpson 或 Runge-Kutta 格式, 便于处理不连续控制结构和变步长问题。然而, 直接法生成的 NLP 问题通常包含数千个变量和约束, 对通用 NLP 求解器 (如 IPOPT、SNOPT) 的依赖导致计算延迟不可预测且难以在嵌入式环境部署, 因此主要应用于地面离线设计和验证环节。

凸优化方法近年来取得了突破性进展。Açıkmeşe 和 Ploen (2007) 首次提出了无损凸化 (Lossless Convexification) 技术, 通过引入松弛变量将原始非凸推力约束中的禁带结构等价变换为二阶锥约束, 从而将原非凸最优控制问题转化为二阶锥规划 (Second-Order Cone Programming, SOCP) [5]。这一方法的核心在于: 变换后的 SOCP 问题的最优解一定满足原非凸问题的所有约束且目标函数值一致, 不存在松弛间隙——这正是“无损”字的数学含义。在此基础上, Blackmore 等人 (2012) 将无损凸化推广至包含状态约束和障碍规避约束的一般着陆场景, 并在 MSL 任务的 FNPEG (Fuel-Optimal Powered Descent Guidance) 后备制导系统中进行了工程验证 [8]。Liu 等人 (2017) 进一步将 SOCP 方法应用于高超声速再入制导问题, 通过序列凸化 (Sequen-

tial Convex Programming) 将气动系数非线性和再入走廊约束逐次线性化, 每一轮迭代求解一个 SOCP 问题 [9]。此外, Lu (2020) 和 Eren (2017) 分别在火箭垂直回收和卫星编队飞行领域验证了 SOCP 方法的高效性。

1.3 现有工作的局限

尽管上述方法在理论层面取得了丰硕成果, 现有研究在工程实现层面仍存在四个亟待解决的不足。第一, 绝大多数的凸优化求解方案依赖通用建模层——通过 YALMIP 或 CVXPY 等建模工具将 SOCP 问题转化为求解器可接受的标准形式, 该过程引入大量冗余变量和约束。例如, 一个 $N=30$ 步离散化后的手写紧致问题仅需 341 个决策变量, 而通过 CVXPY 建模后传递至求解器的实际变量数膨胀至上千, 且引入大量冗余约束矩阵行, 导致求解效率显著下降。第二, 已有工作通常采用 MOSEK、Gurobi、ECOS 等求解器, 侧重精度验证而缺乏对求解速度至关重要的嵌入式部署适配——ECOS 的 C 语言接口虽已公开, 但鲜有工作研究如何将完整求解流程 (矩阵装配、问题形成、求解调用、控制量提取) 集成于单一 C 函数中, 大多停留在 MATLAB/Python 原型阶段。第三, 鲜有文献系统性地对比多款求解器在同一 SOCP 问题上的收敛一致性——不同求解器适用算法迥异 (ECOS 与 Clarabel 采用内点法、acados 基于 SQP、IPOPT 适用滤线搜索), 但针对动力下降场景的数值鲁棒性定量交叉评估尚属空白。第四, 数值精度方面, 现有文献多采用双精度 (double) 浮点数, 而嵌入式微控制器和部分 FPGA 片上系统仅提供单精度浮点单元 (FPU), 精度损失对燃料消耗和终端状态误差的量化影响尚未被深入分析——简单认为单精度“精度不够缺乏定量依据, 而此工作量化的结论直接影响航天器的处理器和编译器选型决策。

1.4 本文贡献

针对上述问题, 本文在无损凸化的理论框架下, 以火星动力下降轨迹优化为具体场景, 做出了以下四方面贡献。

(1) 手写稀疏 SOCP 矩阵构造。本文摒弃通用建模层, 直接以 C 语言手写稀疏锥矩阵——等式约束矩阵含 733 个非零元, 不等式约束矩阵含 403 个非零元, 各矩阵均采用压缩稀疏列 (CSC) 格式存储, 完全兼容 ECOS 原生接口。该构造方式消除了建模层引入的冗余变量和约束, 使传递至求解器的问题规模降到理论最低。矩阵的构建采用分块宏组装策略, 每一类型约束对应一组 CRS_PUSH 宏调用, 代码可读性与效率兼顾。

(2) 六求解器交叉验证。将同一轨迹优化问题分别使用 ECOS (C 手写版及 Python CVXPY 版)、Clarabel、IPOPT、acados 等六种独立求解方案求解。实验表明, 所有方案在燃料消耗 400.7 kg 这一关键指标上完全一致, 最大偏差小于 0.01 kg, 证明了本文手写稀疏矩阵构造在数值上的正确性以及 SOCP 问题在跨求解器场景下的可复现性。

(3) 嵌入式 C 语言实现。将 SOCP 求解全流程——矩阵装配、ECOS 上下文初始化、问题求解、控制量提取——全部抽象为轻量级 C 函数接口, 不依赖任何第三方运行时库。实测单次求解约 8 s (@ 1.5 GHz ARM Cortex-A72), 在动力下降段的典型机载计算资源下可实现 10 Hz 的轨迹重规划频率, 满足实时制导需求。

(4) 数值精度系统分析。对比了单精度 (float) 和双精度 (double) 浮点数在燃料消耗、推力剖面、着陆点误差三个维度上的差异。实验数据显示, Clarabel 单精度实现在标准参数配置下

燃料消耗偏差高达 375% (1880+ kg vs 400.7 kg), 根因为问题数值动态范围达 $10^{6.6}$ ——位置坐标量级为 2000 m, 而质量对数 $\exp(-z)$ 量级仅为 0.0005, 单精度浮点数的 7 位有效数字不足以支撑条件数约 10^8 的 KKT 系统求解。本文进一步分析了 C 手写版在单双精度混用模式下的精度损失剖面, 量化了各约束残差随浮点字长的衰减规律。该分析为嵌入式航天处理器中浮点类型选型和编译器优化选项设定提供了定量依据。

1.5 论文组织

本文余下章节组织如下。第 2 章建立火星动力下降轨迹优化的数学模型, 包括六自由度动力学方程、非凸推力约束、下滑角约束和终端状态约束, 并通过质量对数变换 ($\ln m$) 和无损凸化技术将原问题等价转化为 SOCP 标准形式。第 3 章详细阐述稀疏锥矩阵的手工构造方法, 给出等式约束矩阵 (31 行、341 列、733 非零元) 和不等式约束矩阵 (279 行、341 列、403 非零元) 的维度、稀疏模式和 CSC 格式生成细节。第 4 章介绍六款求解器 (ECOS C 手写版、CVXPY+ECOS、CVXPY+Clarabel、CasADi+IPOPT、acados SQP、C 自动版) 的配置与调用流程, 呈现交叉验证结果并分析各求解器的收敛行为和数值稳定性差异。第 5 章围绕嵌入式 C 语言实现讨论代码架构设计、内存管理模式 (零动态分配、栈上定长数组) 和实时性能基准测试 ($8 \mu\text{s}/\text{solve}$)。第 6 章开展系统的数值精度对比实验, 量化单精度与双精度浮点数在燃料消耗、终端位置和推力剖面三个维度上的偏差, 结合问题的数值条件数给出精度选型建议。第 7 章总结全文工作并提出未来研究方向。

2 问题描述与建模

2.1 坐标系定义

本文采用火星固连坐标系 (Mars-Centered, Mars-Fixed, MCMF) 描述着陆器的运动状态。坐标系原点位于火星质心, x 轴沿火星径向方向指向上方 (远离火星表面), y 轴和 z 轴位于水平面内, 构成右手直角坐标系。在该坐标系下, 着陆器的位置向量记为 $\mathbf{r} = [r_x, r_y, r_z]^\top$, 其中 r_x 表示着陆器距火星参考球面的径向高度, r_y 和 r_z 表示水平方向的位置分量。速度向量记为 $\mathbf{v} = [v_x, v_y, v_z]^\top$, 分别对应各轴方向的速度分量。

在动力下降段, 着陆器距离火星表面仅数千米, 相对于火星半径的尺度而言可近似认为重力加速度为常值。火星表面平均重力加速度为 $g = 3.7114 \text{ m/s}^2$, 方向沿 x 轴负方向 (指向火星表面)。由于动力下降段时间短 ($t_f = 81 \text{ s}$)、运动范围小, 忽略火星自转效应和科里奥利力的影响, 固连坐标系可视为惯性系。

2.2 物理参数

本文所研究的火星动力下降段着陆器物理参数如表1所示。

表 1: 火星动力下降段物理参数

符号	物理意义	数值
g	火星表面重力加速度	3.7114 m/s^2
I_{sp}	发动机比冲	225 s
m_0	初始总质量	1905 kg
m_{dry}	干重 (结构质量)	1505 kg
T_{max}	单台发动机最大推力	3100 N
T_2	姿控可用最大推力 ($0.8 T_{\text{max}}$)	2480 N
n_T	发动机台数	6
ϕ	发动机安装倾角	27°
θ	最大下滑角	86°
N	离散控制段数	30
t_f	给定着陆时间	81 s
$\mathbf{r}_0$	初始位置 $[r_{x0}, r_{y0}, r_{z0}]^\top$	$[1500, 0, 2000]^\top \text{ m}$
$\mathbf{v}_0$	初始速度 $[v_{x0}, v_{y0}, v_{z0}]^\top$	$[-75, 0, 100]^\top \text{ m/s}$

其中, g 为火星表面重力加速度, 取值 3.7114 m/s^2 约为地球重力加速度的 0.378 倍。 I_{sp} 为发动机比冲, 表征推进系统的燃料效率, 取值 225 s 对应于典型液体燃料发动机的性能水平。 m_0 为初始总质量, m_{dry} 为干重 (结构质量), 二者之差 $\Delta m = 400 \text{ kg}$ 即为可用燃料质量。 T_{max} 为单台发动机最大推力, 而 $T_2 = 0.8 T_{\text{max}}$ 为扣除 20% 姿控余量后可用于动力下降的推力上界——这 20% 的余量确保姿态控制系统在动力下降过程中有足够的力矩裕度。 $n_T = 6$ 为发动机台数, 均布于着陆器底部。 $\phi = 27^\circ$ 为发动机喷管轴线与着陆器中心轴线之间的安装倾角, 该倾斜布局使得发动机在提供轴向推力的同时具备水平方向推力分量, 从而实现对水平速度的直接控制。 $\theta = 86^\circ$ 为最大允许下滑角, 保证着陆轨迹近乎垂直, 防止着陆器因水平速度过大而倾覆。 $N = 30$ 为用于数值求解的离散控制段数, $t_f = 81 \text{ s}$ 为给定的着陆时间。

初始位置 $\mathbf{r}_0 = [1500, 0, 2000]^\top \text{ m}$ 意味着着陆器在动力下降开始时位于目标着陆点正上方 1500 m , 同时在 z 方向有 2000 m 的水平偏移。初始速度 $\mathbf{v}_0 = [-75, 0, 100]^\top \text{ m/s}$ 表明着陆器已有 75 m/s 的竖直向下速度分量和 100 m/s 的 z 方向水平速度。

2.3 三自由度动力学方程

在火星固连坐标系下, 着陆器被视为变质量质点, 其三自由度动力学方程由牛顿第二定律和变质量系统的质量守恒律推导得到。连续时间动力学方程组为

$$\dot{\mathbf{r}} = \mathbf{v}, \quad (1)$$

$$\dot{\mathbf{v}} = \mathbf{g} + \frac{\mathbf{T}_{\text{total}}}{m}, \quad (2)$$

$$\dot{m} = -\alpha \|\mathbf{T}_{\text{total}}\|, \quad (3)$$

其中 $\mathbf{T}_{\text{total}} = [T_x, T_y, T_z]^\top$ 为 n_T 台发动机产生的总推力向量, $\mathbf{g} = [-g, 0, 0]^\top$ 为重力加速度向量, m 为着陆器当前时刻的瞬时质量。

式(1)是位置与速度之间的运动学关系，式(2)是牛顿第二定律的矢量形式——着陆器加速度由重力项和控制加速度项（推力除以质量）两部分构成。式(3)为质量消耗方程，描述了推进剂消耗速率与推力幅值之间的正比关系。

系数 α 为单位推力幅值对应的质量消耗率。根据火箭发动机比冲定义，推力幅值与推进剂质量流率的关系为

$$\|\mathbf{T}_{\text{total}}\| = n_T I_{\text{sp}} g_0 (-\dot{m}/n_T) \cos \phi = I_{\text{sp}} g_0 (-\dot{m}) \cos \phi, \quad (4)$$

其中 $g_0 = 9.80665 \text{ m/s}^2$ 为地球标准重力加速度（用于比冲定义）， $\cos \phi$ 因子来源于发动机推力沿轴向的分量。整理得

$$\dot{m} = -\frac{\|\mathbf{T}_{\text{total}}\|}{I_{\text{sp}} g_0 \cos \phi} \triangleq -\alpha \|\mathbf{T}_{\text{total}}\|, \quad \alpha = \frac{1}{I_{\text{sp}} g_0 \cos \phi}. \quad (5)$$

该定义与代码实现 [10] [5] 一致（见 `mars_params.py: alpha = 1.0/(I_sp * g_earth * cos(phi))`）。

将式(2)按各轴分量展开，得到三个标量微分方程：

$$\dot{v}_x = -g + \frac{T_x}{m}, \quad (6)$$

$$\dot{v}_y = \frac{T_y}{m}, \quad (7)$$

$$\dot{v}_z = \frac{T_z}{m}. \quad (8)$$

式(6)中重力加速度 g 带负号，代表重力指向火星表面（即 x 轴负方向）。着陆器要想减速并最终悬浮于目标点，必须依靠推力分量 T_x 提供足够大的向上的加速度，以抵消重力并产生减速效果。当 $T_x > mg$ 时，着陆器在 x 方向获得向上的净加速度， $|v_x|$ 逐渐减小至零。 y 和 z 方向无重力项，水平面的运动加速度完全由推力分量决定。

从式(2)对时间积分，可以得到速度增量的近似表达式：

$$\mathbf{v}(t) - \mathbf{v}_0 = \mathbf{g} t + \int_0^t \frac{\mathbf{T}_{\text{total}}(\tau)}{m(\tau)} d\tau. \quad (9)$$

可见着陆器在动力下降过程中需要克服重力作用，同时通过调节推力矢量补偿初始速度偏差，最终达到终端零速度的条件。

2.4 控制量与推力矢量关系

2.4.1 发动机配置

着陆器底部均布安装 $n_T = 6$ 台液体火箭发动机，每台发动机的推力大小可独立调节，且具备深度节流能力。各发动机的安装轴线与着陆器中心轴线（ x 轴）之间具有相同的安装倾角 $\phi = 27^\circ$ ，即每台发动机的推力方向均偏离 x 轴一个固定角度 ϕ ，且在水平面内均匀分布，相邻两台发动机之间夹角为 60° 。

设第 i 台发动机的推力大小为 T_i ($i = 1, 2, \dots, n_T$)，其推力向量在火星固连坐标系中的分解为

$$\mathbf{T}_i = T_i \begin{bmatrix} \cos \phi \\ \sin \phi \cos \psi_i \\ \sin \phi \sin \psi_i \end{bmatrix}, \quad (10)$$

其中 $\psi_i = (i - 1) \times 60^\circ$ 为第 i 台发动机轴线在水平面内投影的方位角。各台发动机的方位角具体为

$$\psi_i \in \{0^\circ, 60^\circ, 120^\circ, 180^\circ, 240^\circ, 300^\circ\}. \quad (11)$$

n_T 台发动机的总推力向量为各台推力向量之和:

$$\mathbf{T}_{\text{total}} = \sum_{i=1}^{n_T} \mathbf{T}_i = \begin{bmatrix} \sum_{i=1}^{n_T} T_i \cos \phi \\ \sum_{i=1}^{n_T} T_i \sin \phi \cos \psi_i \\ \sum_{i=1}^{n_T} T_i \sin \phi \sin \psi_i \end{bmatrix}. \quad (12)$$

2.4.2 加速度控制量

为简化控制问题的表述, 引入加速度控制量 $\mathbf{u} = [u_x, u_y, u_z]^\top$, 其定义为总推力向量除以当前质量:

$$\mathbf{u} = \frac{\mathbf{T}_{\text{total}}}{m}. \quad (13)$$

将式(12)代入式(13), 可得加速度控制量的各轴分量表达式为

$$u_x = \frac{\cos \phi}{m} \sum_{i=1}^{n_T} T_i, \quad (14)$$

$$u_y = \frac{\sin \phi}{m} \sum_{i=1}^{n_T} T_i \cos \psi_i, \quad (15)$$

$$u_z = \frac{\sin \phi}{m} \sum_{i=1}^{n_T} T_i \sin \psi_i. \quad (16)$$

当 6 台发动机推力不等时, 总推力矢量的方向可在一定范围内连续变化。令 $\mathbf{e}_i = [\cos \phi, \sin \phi \cos \psi_i, \sin \phi \sin \psi_i]^\top$, 则总推力矢量可控空间为

$$\mathbf{T}_{\text{total}} \in \left\{ \sum_{i=1}^{n_T} T_i \mathbf{e}_i : T_i \in [T_{\min}, T_{\max}] \right\}. \quad (17)$$

由于 $n_T = 6$ 且 $\mathbf{e}_i$ 在三维空间中张成了多面锥, 总推力矢量的可达集是一个以 x 轴为对称轴的多边形锥体。对于本文所考虑的燃料最优轨迹规划问题, 进一步假设各发动机同步调节, 即每台发动机的推力大小相等:

$$T_1 = T_2 = \dots = T_{n_T} = T. \quad (18)$$

此时总推力矢量方向由几何配置唯一确定, 各轴分量简化为

$$T_x = n_T T \cos \phi, \quad (19)$$

$$T_y = n_T T \sin \phi \cdot \frac{1}{6} \sum_{i=1}^6 \cos \psi_i = 0, \quad (20)$$

$$T_z = n_T T \sin \phi \cdot \frac{1}{6} \sum_{i=1}^6 \sin \psi_i = 0. \quad (21)$$

由于余弦和正弦在完整圆周上的对称性, $\sum \cos \psi_i = \sum \sin \psi_i = 0$, 均布推力时水平分量互相抵消, 总推力方向仅沿 x 轴方向。但在非同步调节的一般情况下, 通过调制各台发动机的推力差异, 可以在水平面内产生净推力分量, 从而实现水平方向的速度控制。本文采用等效总推力向量 $\mathbf{T} = [T_x, T_y, T_z]^\top$ 作为实际设计变量, 直接表示总推力在各轴上的分量, 令加速度控制量 $\mathbf{u}$ 为

$$u_x = \frac{T_x}{m}, \quad u_y = \frac{T_y}{m}, \quad u_z = \frac{T_z}{m}. \quad (22)$$

将式(13)与(5)代入质量消耗方程(3), 得到以加速度控制量表示的质量变化率:

$$\dot{m} = -\alpha m \|\mathbf{u}\|. \quad (23)$$

至此, 以 $\mathbf{r}$ 、 $\mathbf{v}$ 、 m 为状态变量, 以 $\mathbf{u}$ 为控制变量的连续时间动力学方程组综合如下:

$$\dot{\mathbf{r}} = \mathbf{v}, \quad (24)$$

$$\dot{\mathbf{v}} = \mathbf{g} + \mathbf{u}, \quad (25)$$

$$\dot{m} = -\alpha m \|\mathbf{u}\|. \quad (26)$$

式(26)是微分方程中唯一的非线性项, 其右端包含了状态 m 与控制量 $\mathbf{u}$ 的耦合乘积 $\|\mathbf{u}\|$, 是该最优控制问题非线性的主要来源, 也是后续松弛和凸化处理的关键对象。

2.5 约束条件

2.5.1 推力幅值约束

每台液体火箭发动机的推力大小受到物理限制, 只能在最小推力 $T_{\min}$ 和最大推力 $T_{\max}$ 之间连续调节:

$$T_{\min} \leq T_i \leq T_{\max}, \quad i = 1, 2, \dots, n_T. \quad (27)$$

需要说明的是, 对于液体火箭发动机, $T_{\min}$ 通常不为零, 而是受到燃烧室稳定工作条件的限制 (典型值为 30% 额定推力)。但在动力下降段的数学规划中, 通常不对 $T_{\min}$ 施加严格的下界约束, 而是依靠最优解的自然特性保证发动机工作在合理区间。

考虑工程实际, 需要预留 20% 的推力用于姿态控制系统, 因此实际可用于动力下降的可用最大推力为

$$T_2 = 0.8 T_{\max} = 2480 \text{ N}. \quad (28)$$

由 n_T 台发动机的总推力幅值 $\|\mathbf{T}_{\text{total}}\|$ 满足的约束为

$$0 \leq \|\mathbf{T}_{\text{total}}\| \leq n_T T_2. \quad (29)$$

将式(13)代入上式, 得到关于加速度控制量 $\mathbf{u}$ 的约束:

$$0 \leq \|\mathbf{u}\| \leq \frac{n_T T_2}{m}. \quad (30)$$

注意到该约束的上界与状态变量 m 有关, 属于状态相关的控制约束 (state-dependent control constraint)。随着推进剂的消耗, m 逐渐减小, 约束上界 $\frac{n_T T_2}{m}$ 逐渐增大, 意味着后期可用加速度更大。

2.5.2 下滑角约束

为保证着陆器以近乎垂直的姿态安全着陆，避免因水平速度过大导致着陆时倾覆，需对飞行轨迹的下滑角施加约束。下滑角定义为速度向量与水平面之间的夹角 γ ，满足

$$\tan \gamma = \frac{-v_x}{\sqrt{v_y^2 + v_z^2}}. \quad (31)$$

该约束等价于要求速度方向始终保持在以 x 轴为对称轴的一个倒锥体内。在仅给定初始和终端位置的情况下，更常用的是路径约束形式——位置空间中的锥形约束，以确保着陆器始终保持在安全走廊内：

$$\sqrt{r_y^2 + r_z^2} \leq r_x \tan \theta, \quad (32)$$

其中 $\theta = 86^\circ$ 为最大允许下滑角， $\tan 86^\circ \approx 14.30$ 。该约束的几何意义如图1所示，它定义了以 x 轴线为中心轴、半顶角为 86° 的倒锥形安全走廊，着陆器的三维位置 $[r_x, r_y, r_z]$ 必须在锥体内部。

将 $\theta = 86^\circ$ 代入式(32)，得到具体的约束不等式为

$$\sqrt{r_y^2 + r_z^2} \leq 14.30 r_x. \quad (33)$$

对于初始位置 $r_{x0} = 1500$ m，水平方向最大允许偏移量为 $14.30 \times 1500 \approx 21450$ m——远大于实际的初始水平偏移 2000 m。因此，该约束在动力下降的前期和中段并不活跃，它的主要作用是约束终端段的飞行姿态：随着 r_x 趋近于零，允许的水平偏移范围也迅速缩小，保证着陆器最终以近似垂直的状态抵达着陆点。

在优化问题的数值求解中，下滑角约束(32)是一个典型的二阶锥约束，可写为以下标准形式：

$$\|[r_y, r_z]^\top\| \leq r_x \tan \theta. \quad (34)$$

2.5.3 质量约束

着陆器在动力下降过程中的质量受物理极限的约束，其取值范围为

$$m_{\text{dry}} \leq m(t) \leq m_0, \quad \forall t \in [0, t_f]. \quad (35)$$

其中 $m_{\text{dry}} = 1505$ kg 为着陆器结构干重，包含着陆器本体结构、有效载荷和未消耗的残余推进剂的质量。 $m_0 = 1905$ kg 为动力下降开始时的初始总质量，由进入段末端的状态确定。质量差 $\Delta m = m_0 - m_{\text{dry}} = 400$ kg 即为可用于动力下降的燃料总储备。在燃料最优轨迹规划中，终端质量 $m(t_f)$ 越接近 m_{dry} ，意味着燃料消耗越接近于可用燃料储备。

2.5.4 边界条件

动力下降段的初始状态由导航系统在进入段末端给出的测量值确定，是固定的已知量：

$$\mathbf{r}(0) = \mathbf{r}_0 = [1500, 0, 2000]^\top \text{ m}, \quad (36)$$

$$\mathbf{v}(0) = \mathbf{v}_0 = [-75, 0, 100]^\top \text{ m/s}, \quad (37)$$

$$m(0) = m_0 = 1905 \text{ kg}. \quad (38)$$

终端状态要求着陆器在给定着陆时刻 $t_f = 81$ s 以零速度精确到达目标着陆点（火星固连坐标系原点）：

$$\mathbf{r}(t_f) = \mathbf{0}, \quad (39)$$

$$\mathbf{v}(t_f) = \mathbf{0}. \quad (40)$$

终端质量不作约束，由动力学方程自然演化确定，但自动满足 $m(t_f) \geq m_{\text{dry}}$ 。

2.6 燃料最优目标函数

在火星动力下降任务中，推进剂是最宝贵的资源。有效载荷质量与燃料消耗直接相关，因此燃料最优化是轨迹规划的核心目标。

燃料消耗总量等于初始质量减去终端质量：

$$J = m_0 - m(t_f). \quad (41)$$

利用式(3)将终端质量表示为对质量变化率的积分：

$$m(t_f) = m_0 + \int_0^{t_f} \dot{m}(t) dt = m_0 - \int_0^{t_f} \alpha \|\mathbf{T}_{\text{total}}(t)\| dt. \quad (42)$$

代入 J 的表达式，得到

$$J = \int_0^{t_f} \alpha \|\mathbf{T}_{\text{total}}(t)\| dt. \quad (43)$$

由于 α 为正常数，燃料最优等价于最小化总推力对时间的积分，即推力总冲 (total impulse)：

$$\min J = \int_0^{t_f} \|\mathbf{T}_{\text{total}}(t)\| dt. \quad (44)$$

此目标函数的物理意义直观明确：减小任意时刻的推力幅值或缩短推力作用时间均可降低燃料消耗。但由于终端时间 t_f 固定，质心在缩短推力时间，这意味着需要增大推力幅值以提供足够的减速度，从而产生了推力大小与作用时间之间的折中关系。

将式(13)代入式(44)，得到以加速度控制量 $\mathbf{u}$ 表示的目标函数：

$$\min J = \int_0^{t_f} m(t) \|\mathbf{u}(t)\| dt. \quad (45)$$

该表达式中含有状态量 $m(t)$ 与控制量 $\mathbf{u}(t)$ 的乘积项，进一步体现了目标函数的非线性特性。

2.7 Bolza 型最优控制问题

综合上述动力学方程、约束条件和目标函数，火星动力下降燃料最优轨迹规划问题最终可表述为以下连续时间最优控制问题的标准 Bolza 形式。

状态变量：

$$\mathbf{x}(t) = [\mathbf{r}(t)^\top, \mathbf{v}(t)^\top, m(t)]^\top \in \mathbb{R}^7. \quad (46)$$

控制变量：

$$\mathbf{u}(t) = [u_x(t), u_y(t), u_z(t)]^\top \in \mathbb{R}^3. \quad (47)$$

动力学方程：

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \mathbf{v} \\ \mathbf{g} + \mathbf{u} \\ -\alpha m \|\mathbf{u}\| \end{bmatrix}. \quad (48)$$

路径约束：

$$\|\mathbf{u}(t)\| \leq \frac{n_T T_2}{m(t)}, \quad (49)$$

$$\sqrt{r_y^2 + r_z^2} \leq r_x \tan \theta, \quad (50)$$

$$m_{\text{dry}} \leq m(t) \leq m_0. \quad (51)$$

边界条件:

$$\mathbf{x}(0) = [\mathbf{r}_0^\top, \mathbf{v}_0^\top, m_0]^\top, \quad (52)$$

$$\mathbf{r}(t_f) = \mathbf{0}, \quad \mathbf{v}(t_f) = \mathbf{0}. \quad (53)$$

目标函数 (Lagrange 型) :

$$\min J = \int_0^{t_f} \mathcal{L}(\mathbf{x}, \mathbf{u}) dt, \quad (54)$$

其中 Lagrange 函数为

$$\mathcal{L}(\mathbf{x}, \mathbf{u}) = m(t) \|\mathbf{u}(t)\|. \quad (55)$$

由于式(54)中仅含有运行代价积分项而终端代价为零, 该问题在严格意义上属于 Lagrange 型最优控制问题, 但其结构可以视为 Bolza 型 (含积分代价 + 终端代价) 的一个特例。

式(48)–(54)构成完整的连续时间最优控制问题。利用庞特里亚金极小值原理 (Pontryagin's Minimum Principle) 分析该问题的结构, 可揭示其最优解的理论特征。定义 Hamiltonian 函数:

$$\mathcal{H}(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda}) = m\|\mathbf{u}\| + \boldsymbol{\lambda}_r^\top \mathbf{v} + \boldsymbol{\lambda}_v^\top (\mathbf{g} + \mathbf{u}) - \lambda_m \alpha m\|\mathbf{u}\|, \quad (56)$$

其中 $\boldsymbol{\lambda} = [\boldsymbol{\lambda}_r^\top, \boldsymbol{\lambda}_v^\top, \lambda_m]^\top$ 为协态向量 (costate)。根据极小值原理, 最优控制量 $\mathbf{u}^*$ 在每一时刻应最小化 Hamiltonian 函数:

$$\mathbf{u}^* = \arg \min_{\|\mathbf{u}\| \leq n_T T_2 / m} \left[m\|\mathbf{u}\| + \boldsymbol{\lambda}_v^\top \mathbf{u} - \lambda_m \alpha m\|\mathbf{u}\| \right]. \quad (57)$$

整理关于 $\|\mathbf{u}\|$ 的系数可知, 最优控制的开关结构由协态的演化轨迹决定。这一分析揭示了最优推力幅值和方向的控制律结构, 为后续章节直接法的离散化方案和松弛策略提供了理论参照。

该问题具有以下数学特征:

1. **非线性动力学:** 式(48)中质量变化率方程包含状态与控制量的耦合项 $m\|\mathbf{u}\|$, 使系统呈现本质非线性;
2. **非凸控制约束:** 式(49)中控制约束的上界依赖于状态 $m(t)$, 属于状态相关的控制约束 (state-dependent constraint), 导致可行域非凸;
3. **二阶锥约束:** 下滑角约束(50)是典型的二阶锥约束, 形式为 $\|[\cdot]\| \leq \text{linear expression}$;
4. **自由终端质量:** 终端质量由动力学自然确定, 仅受不等式下界约束 $m(t_f) \geq m_{\text{dry}}$;
5. **多约束耦合:** 路径约束与动力学方程之间存在复杂的相互耦合关系, 例如推力上界约束的松紧程度取决于当前质量, 而质量又由推力积分决定。

由于上述非线性和多约束耦合特性, 该问题的解析求解极为困难。传统间接法虽然能通过求解两点边值问题得到最优解, 但面对多约束和状态相关约束时, 初值猜测极为敏感, 收敛十分困难。直接法将连续时间控制问题进行离散化, 将其转化为有限维参数优化问题, 具有更强的鲁棒性, 是工程实践中的主流方法。

本文后续章节将基于凸优化理论, 通过对上述最优控制问题进行凸化松弛和离散化处理, 将其转化为可高效求解的二阶锥规划 (Second-Order Cone Programming, SOCP) 问题。转化路径包含三个关键步骤。

步骤一：变量替换。引入新状态变量 $z(t) = \ln m(t)$, 对式(26)进行对数变换:

$$\dot{z} = \frac{\dot{m}}{m} = -\alpha \|\mathbf{u}\|. \quad (58)$$

变换后, 质量动力学方程变为关于 z 的线性微分方程, 消除了原方程中状态与控制量的乘积耦合。同时, 质量约束(51)转化为 z 的线性约束:

$$\ln m_{\text{dry}} \leq z(t) \leq \ln m_0. \quad (59)$$

步骤二：约束松弛。推力上界约束(49)在变量替换后变为

$$\|\mathbf{u}(t)\| \leq n_T T_2 e^{-z(t)}, \quad (60)$$

该约束的右端项 $e^{-z(t)}$ 关于 z 是凸函数, 但以 z 为自变量的形式使得约束集仍然非凸。通过引入松弛变量 $\sigma(t) \geq \|\mathbf{u}(t)\|$ 并利用 e^{-z} 的凸性进行序列线性化或直接写作 SOC 约束, 可将该非凸约束转化为凸约束近似。同时需要证明松弛的最优性, 即最优解处 $\sigma(t) = \|\mathbf{u}(t)\|$ 严格成立。

步骤三：直接转录。将时间区间 $[0, t_f]$ 等分为 N 个子区间, 在每个子区间上采用零阶保持 (Zero-Order Hold, ZOH) 离散化策略, 即控制量在每一段内保持为常值。利用梯形法或欧拉法对微分方程进行数值积分, 得到一组等式约束 (即动力学配点约束)。最终获得一个标准 SOCP 问题, 可直接利用内点法求解器 (如 ECOS、Clarabel 等) 高效求解。

上述三个步骤构成完整的问题转化链条。在第3章中, 将详细阐述每一步的数学推导细节, 包括松弛紧性的严格证明和离散化误差分析。

2.8 无量纲化处理

为改善数值求解的收敛性和计算精度, 对上述最优控制问题中的状态变量和控制变量进行无量纲化处理。引入无量纲基准量: 位置基准 $r_{\text{ref}} = r_{x0} = 1500 \text{ m}$, 质量基准 $m_{\text{ref}} = m_0 = 1905 \text{ kg}$, 时间基准 $t_{\text{ref}} = 1 \text{ s}$ 。无量纲变量定义为:

$$\bar{\mathbf{r}} = \frac{\mathbf{r}}{r_{\text{ref}}}, \quad \bar{\mathbf{v}} = \frac{\mathbf{v} t_{\text{ref}}}{r_{\text{ref}}}, \quad \bar{m} = \frac{m}{m_{\text{ref}}}, \quad \bar{t} = \frac{t}{t_{\text{ref}}}, \quad (61)$$

$$\bar{\mathbf{u}} = \frac{\mathbf{u} t_{\text{ref}}^2}{r_{\text{ref}}}, \quad \bar{g} = \frac{g t_{\text{ref}}^2}{r_{\text{ref}}}, \quad \bar{\alpha} = \frac{\alpha m_{\text{ref}} r_{\text{ref}}}{t_{\text{ref}}^2}. \quad (62)$$

代入式(24)–(26), 得到无量纲动力学方程:

$$\frac{d\bar{\mathbf{r}}}{d\bar{t}} = \bar{\mathbf{v}}, \quad (63)$$

$$\frac{d\bar{\mathbf{v}}}{d\bar{t}} = \bar{\mathbf{g}} + \bar{\mathbf{u}}, \quad (64)$$

$$\frac{d\bar{m}}{d\bar{t}} = -\bar{\alpha} \bar{m} \|\bar{\mathbf{u}}\|. \quad (65)$$

无量纲化处理后, 位置变量量级约为 $O(1)$, 速度变量量级约为 $O(10^{-1})$, 质量变量量级为 $O(1)$, 控制变量量级约为 $O(10^{-3})$ 。各变量数值范围得到大幅压缩, 与求解器内部默认数值容差 (通常在 10^{-8} 到 10^{-6} 之间) 相匹配, 有效提高了数值优化的收敛性和计算精度。此外, 无量纲化还消除了物理单位不统一可能引入的数值误差。本文后续章节的数值求解均采用无量纲化后的变量。

3 无损凸化与离散化

本章是全文的核心理论章节，系统地阐述将非凸燃料最优着陆控制问题转化为二阶锥规划 (SOCP) 的完整过程。首先分析原始问题中非线性和非凸性的来源与数学本质，然后通过变量变换将非线性动力学转化为线性形式，对非凸约束进行松弛与线性化处理，证明各变换环节的凸性保持性质，最后采用直接转录法对连续时间问题进行离散化，得到标准形式的离散 SOCP 问题。本章还简要引用 Lossless Convexification 理论来保证松弛的精确性。

3.1 凸化动机与问题描述

3.1.1 原始最优控制问题

考虑三维空间中的火箭动力着陆问题。定义状态变量为位置矢量 $\mathbf{r}(t) \in \mathbb{R}^3$ 、速度矢量 $\mathbf{v}(t) \in \mathbb{R}^3$ 和质量 $m(t) \in \mathbb{R}_+$ ，控制变量为推力矢量 $\mathbf{T}(t) \in \mathbb{R}^3$ 。在常值重力场中，系统的连续时间动力学方程为：

$$\dot{\mathbf{r}} = \mathbf{v}, \quad (66)$$

$$\dot{\mathbf{v}} = \mathbf{g} + \frac{\mathbf{T}}{m}, \quad (67)$$

$$\dot{m} = -\alpha \|\mathbf{T}\|, \quad (68)$$

其中 $\mathbf{g} = [-g, 0, 0]^\top$ 为重力加速度矢量 ($g = 3.7114 \text{ m/s}^2$)， $\alpha = 1/(I_{sp}g_0 \cos \phi)$ 为推力比冲相关的常数 (I_{sp} 为比冲， g_0 为海平面重力加速度)。初始条件与终端条件分别为：

$$\mathbf{r}(0) = \mathbf{r}_0, \quad \mathbf{v}(0) = \mathbf{v}_0, \quad m(0) = m_0, \quad (69)$$

$$\mathbf{r}(t_f) = \mathbf{r}_f, \quad \mathbf{v}(t_f) = \mathbf{v}_f, \quad (70)$$

其中 t_f 为总飞行时间， $\mathbf{r}_f$ 为目标着陆点位置， $\mathbf{v}_f = \mathbf{0}$ 为着陆速度（软着陆条件）。推力幅值受到物理限制：

$$T_{\min} \leq \|\mathbf{T}(t)\| \leq T_{\max}, \quad \forall t \in [0, t_f], \quad (71)$$

其中 $T_{\min} > 0$ 和 $T_{\max}$ 分别为推力器的最小和最大推力幅值。此外，飞行轨迹需满足下滑角约束，以确保飞行器沿安全的走廊接近着陆点，避免过度水平位移导致撞地：

$$\|[r_y(t), r_z(t)]^\top\| \leq r_x(t) \tan \theta, \quad \forall t \in [0, t_f], \quad (72)$$

其中 $\theta \in (0, \pi/2)$ 为最大允许下滑角。燃料最优目标为最大化终端质量，等价于最小化燃料消耗：

$$J = -m(t_f) = -e^{z(t_f)} = \int_0^{t_f} \alpha \|\mathbf{T}(t)\| dt. \quad (73)$$

上述最优控制问题（记为 $\mathcal{P}_0$ ）中的非线性和非凸性来源于以下几个方面，需要通过凸化技术逐一处理。

3.1.2 质量动力学方程的非线性

方程 (68) 中, 质量变化率 $\dot{m}$ 与推力范数 $\|\mathbf{T}\|$ 成比例, 而推力矢量本身也出现在方程 (67) 中并除以当前质量 m 。具体而言, 动力学方程中存在 $\mathbf{T}/m$ 这类状态与控制变量的耦合项, 且质量方程 $\dot{m} = -\alpha\|\mathbf{T}\|$ 本身关于状态 m 和控制 $\mathbf{T}$ 都是非线性的。这种耦合使得 $\mathcal{P}_0$ 不属于凸优化问题的范畴。

从凸优化的角度看, 标准凸最优控制问题要求动力学方程关于状态和控制是线性的 (或至少是仿射的), 且约束集合是凸集。然而 $\mathbf{T}/m$ 项显然不是状态和控制变量的仿射函数, 因此必须通过适当的变量变换将其消除。

3.1.3 推力幅值约束的非凸性

推力幅值约束包含上界和下界两部分, 它们的凸性性质截然不同:

- **上界约束** $\|\mathbf{T}\| \leq T_{\max}$: 这是凸约束。因为 $\mathbf{T} \mapsto \|\mathbf{T}\|$ 是凸函数 (满足三角不等式和正齐次性), 其下水平集 $\{\mathbf{T} \in \mathbb{R}^3 \mid \|\mathbf{T}\| \leq T_{\max}\}$ 是凸集——具体而言, 这是一个半径为 $T_{\max}$ 的三维球体 (包含内部区域)。
- **下界约束** $\|\mathbf{T}\| \geq T_{\min}$: 这是非凸约束。集合 $\{\mathbf{T} \in \mathbb{R}^3 \mid \|\mathbf{T}\| \geq T_{\min}\}$ 是三维球体的补集, 即半径为 $T_{\min}$ 的球体外部区域。对于该集合中的任意两点, 连接它们的线段可能穿过球体内部 (不满足约束的区域), 因此该集合不满足凸集的定义。这是原始问题非凸性的核心来源之一。

从数学上严格论证: 若 $\mathbf{T}_1, \mathbf{T}_2$ 满足 $\|\mathbf{T}_1\| \geq T_{\min}$ 且 $\|\mathbf{T}_2\| \geq T_{\min}$, 取 $\lambda \in (0, 1)$, 则 $\mathbf{T}_3 = \lambda\mathbf{T}_1 + (1 - \lambda)\mathbf{T}_2$ 不一定满足 $\|\mathbf{T}_3\| \geq T_{\min}$ 。反例可在二维情况下轻松构造: 取 $\mathbf{T}_1 = (T_{\min}, 0)$, $\mathbf{T}_2 = (-T_{\min}, 0)$, 则 $\|\mathbf{T}_1\| = \|\mathbf{T}_2\| = T_{\min}$, 但 $\mathbf{T}_1 + \mathbf{T}_2 = 0$, 其范数为零, 不满足下界约束。这直观地说明了推力下界约束的非凸本质。

由于推力器存在最小可靠工作推力 ($T_{\min} > 0$), 该约束不可忽略。在后续章节中, 我们将通过变量变换和一阶泰勒展开线性化来克服这一非凸性。

3.1.4 非凸约束的数学模型整合

将全部约束以集合形式表述, 原始问题 $\mathcal{P}_0$ 的可行域为:

$$\mathcal{F} = \{(\mathbf{r}, \mathbf{v}, m, \mathbf{T}) \mid \text{动力学方程 (66)–(68) 成立}, \quad (74)$$

$$\mathbf{T} \in \{\mathbf{T} \mid T_{\min} \leq \|\mathbf{T}\| \leq T_{\max}\}, \quad (75)$$

$$\| [r_y, r_z] \| \leq r_x \tan \theta, \quad (76)$$

$$\mathbf{r}(0) = \mathbf{r}_0, \mathbf{v}(0) = \mathbf{v}_0, m(0) = m_0, \quad (77)$$

$$\mathbf{r}(t_f) = \mathbf{r}_f, \mathbf{v}(t_f) = \mathbf{v}_f\}. \quad (78)$$

该集合由非凸的推力幅值下界约束和双线性动力学项共同导致非凸性。因此需要系统性的凸化策略, 包括变量变换、约束松弛和线性化近似三个核心步骤, 分别在第 3.2 节至第 3.3 节中详细讨论。

3.2 变量变换

为了消除状态与控制之间的耦合，将动力学转化为仿射形式，引入以下三组变换变量。这些变换在航天最优控制领域具有成熟的理论基础，最早由 Ağıkmeşe 等人（2007）系统应用于火星着陆问题。

3.2.1 加速度控制量

定义加速度控制量 $\mathbf{u}(t) \in \mathbb{R}^3$ 为单位质量上的推力，即推力除以当前质量：

$$\mathbf{u}(t) \triangleq \frac{\mathbf{T}(t)}{m(t)}. \quad (79)$$

则速度动力学方程 (67) 变为线性形式：

$$\dot{\mathbf{v}} = \mathbf{g} + \mathbf{u}. \quad (80)$$

这一变换的关键在于，原本与质量 m 耦合的推力项 $\mathbf{T}/m$ 被统一为新的独立控制变量 $\mathbf{u}$ ，从而消除了状态变量 m 与控制变量 $\mathbf{T}$ 之间的耦合关系。

3.2.2 推力松弛变量

定义标量松弛变量 $\sigma(t) \in \mathbb{R}_+$ ，使其在最优解处等于推力加速度的范数：

$$\sigma(t) \triangleq \frac{\|\mathbf{T}(t)\|}{m(t)} = \|\mathbf{u}(t)\|. \quad (81)$$

在凸化过程中，将上述等式约束松弛为不等式 $\sigma(t) \geq \|\mathbf{u}(t)\|$ ，其精确性由 Lossless Convexification 理论保证（详见第 3.7 节）。这一松弛将非凸的等式约束替换为凸的二阶锥约束，构成了凸化的关键一步。

3.2.3 质量对数变换

定义质量对数状态 $z(t) \in \mathbb{R}$ ：

$$z(t) \triangleq \ln m(t), \quad m(t) = e^{z(t)}. \quad (82)$$

对时间求导并应用链式法则：

$$\dot{z} = \frac{d}{dt} \ln m = \frac{\dot{m}}{m}. \quad (83)$$

代入方程 (68)：

$$\dot{z} = \frac{-\alpha \|\mathbf{T}\|}{m} = -\alpha \frac{\|\mathbf{T}\|}{m} = -\alpha \sigma. \quad (84)$$

因此质量对数动力学成为关于 σ 的线性方程：

$$\dot{z} = -\alpha \sigma. \quad (85)$$

质量对数变换的数学本质是利用了对数函数的微分性质 $d(\ln m)/dt = \dot{m}/m$ ，将 $\dot{m}$ 方程中质量变量 m 从分母位置消除，同时自然地引入了松弛变量 σ 。

3.2.4 变换后的等价动力学系统

经过上述三组变换，原动力学系统 (66)–(68) 等价地转化为：

$$\dot{\mathbf{r}} = \mathbf{v}, \quad (86)$$

$$\dot{\mathbf{v}} = \mathbf{g} + \mathbf{u}, \quad (87)$$

$$\dot{z} = -\alpha\sigma, \quad (88)$$

其中状态变量为 $(\mathbf{r}, \mathbf{v}, z) \in \mathbb{R}^7$ ，控制变量为 $(\mathbf{u}, \sigma) \in \mathbb{R}^4$ 。初始条件相应变为：

$$\mathbf{r}(0) = \mathbf{r}_0, \quad \mathbf{v}(0) = \mathbf{v}_0, \quad z(0) = z_0 \triangleq \ln m_0. \quad (89)$$

终端条件保持不变：

$$\mathbf{r}(t_f) = \mathbf{r}_f, \quad \mathbf{v}(t_f) = \mathbf{v}_f. \quad (90)$$

方程 (86)–(88) 关于状态变量 $(\mathbf{r}, \mathbf{v}, z)$ 和控制变量 $(\mathbf{u}, \sigma)$ 都是**严格线性**的。这为后续的凸优化求解奠定了最关键的基础——线性动力学约束构成一个仿射子空间，是凸集。

3.3 推力约束的凸化

变量变换后，原始的推力幅值约束需要在新变量空间中重新表述并进行凸化处理。

3.3.1 推力上界的二阶锥表示

由 $\mathbf{T} = m\mathbf{u} = e^z\mathbf{u}$ ，推力上界 $\|\mathbf{T}\| \leq T_{\max}$ 等价于：

$$e^z\|\mathbf{u}\| \leq T_{\max}. \quad (91)$$

利用松弛变量 $\sigma \geq \|\mathbf{u}\|$ ，可进一步放松为：

$$e^z\sigma \leq T_{\max} \iff \sigma \leq T_{\max}e^{-z}. \quad (92)$$

结合松弛约束 $\sigma \geq \|\mathbf{u}\|$ ，推力上界的完整凸表述为两个约束的联立：

$$\|\mathbf{u}\| \leq \sigma \leq T_{\max}e^{-z}. \quad (93)$$

其中左侧约束 $\|\mathbf{u}\| \leq \sigma$ 可写为如下标准二阶锥 (Second-Order Cone, SOC) 形式：

$$[\sigma, u_x, u_y, u_z]^\top \in \mathcal{K}_4, \quad (94)$$

其中 $\mathcal{K}_q \triangleq \{\mathbf{x} \in \mathbb{R}^q \mid x_0 \geq \|(x_1, \dots, x_{q-1})\|\}$ 为 q 维二阶锥 (Lorentz 锥)。该锥是凸锥，且为自对偶锥。

3.3.2 推力下界约束的非凸性分析

推力下界 $\|\mathbf{T}\| \geq T_{\min}$ 在新变量空间中等价于：

$$e^z \sigma \geq T_{\min} \iff \sigma \geq T_{\min} e^{-z}. \quad (95)$$

函数 e^{-z} 关于 z 是凸函数（其二阶导数为正）。然而约束 $\sigma \geq T_{\min} e^{-z}$ 要求 σ 大于一个凸函数，这等价于 (z, σ) 空间中凸函数的上方图（epigraph）之补集，因此不是凸约束。更精确地说，集合

$$\{(z, \sigma) \in \mathbb{R}^2 \mid \sigma \geq T_{\min} e^{-z}\} \quad (96)$$

在 (z, σ) 平面上是凸函数 $f(z) = T_{\min} e^{-z}$ 的上方区域，它本身的确是凸集（凸函数的上方图是凸集）。这里需要特别说明：单独看 $\sigma \geq T_{\min} e^{-z}$ 这一约束，它在 (z, σ) 空间中是凸的。问题在于，该约束与动力学约束 (88) 耦合后， z 本身是状态变量，且 z 的演化通过 σ 影响燃料消耗，因此完整的耦合系统在原始变量空间中呈现非凸性。此外，由于 $\sigma \geq \|\mathbf{u}\|$ 和 $e^z \sigma \geq T_{\min}$ 的综合作用，最优解处需要同时满足 σ 不能太大（否则浪费燃料）也不能太小（否则违反推力下界），这种双侧约束配合非线性指数项导致了非凸的可行域拓扑结构。

3.3.3 一阶泰勒展开线性化

为克服上述非凸性，采用序列凸化方法中的核心技术——在参考轨迹附近进行一阶泰勒展开，得到线性近似约束。

首先需构建质量对数的参考轨迹 $z_0(k)$ 。在离散时间框架下 ($k = 0, 1, \dots, N$)，选取飞行器全程以最大推力消耗燃料的极端工况作为参考：

$$z_0(k) = \ln(m_0 - \alpha \rho_2 \cdot k \Delta t), \quad k = 0, 1, \dots, N, \quad (97)$$

其中 $\Delta t = t_f/N$ 为离散时间步长。该参考轨迹的物理含义为：假设飞行器从初始时刻起始终以最大推力 $T_{\max}$ 工作，则质量按 $m(t) = m_0 - \alpha T_{\max} t$ 线性递减，对应的质量对数为 $z(k \Delta t) = \ln(m_0 - \alpha \rho_2 \cdot k \Delta t)$ 。由于实际飞行中推力通常不会始终处于最大值（燃料最优解往往在 $T_{\min}$ 和 $T_{\max}$ 之间切换），该参考轨迹提供了质量对数的最保守估计——实际质量对数 $z(t)$ 应不低于该参考值。

在 z_0 处对 $f(z) = e^{-z}$ 进行一阶泰勒展开，保留前两项：

$$f(z) = e^{-z}, \quad f'(z) = -e^{-z}, \quad (98)$$

$$e^{-z} \approx e^{-z_0} - e^{-z_0}(z - z_0) = e^{-z_0} [1 - (z - z_0)]. \quad (99)$$

该展开式的截断误差为 $O((z - z_0)^2)$ ，在参考轨迹附近具有二阶精度。

将线性化后的 e^{-z} 代入推力下界约束 $\sigma \geq T_{\min} e^{-z}$ 得到：

$$\sigma \geq T_{\min} e^{-z_0} [1 - (z - z_0)]. \quad (100)$$

整理为标准线性不等式（将所有变量项移至左侧，常数项移至右侧）：

$$\sigma + T_{\min} e^{-z_0} (z - z_0 - 1) \geq 0. \quad (101)$$

定义线性化系数 $\mu_1(k)$:

$$\mu_1(k) \triangleq \rho_1 e^{-z_0(k)} = \frac{\rho_1}{m_0 - \alpha \rho_2 \cdot k \Delta t}, \quad k = 0, 1, \dots, N. \quad (102)$$

则下界线性化约束的简洁形式为:

$$\sigma_k + \mu_1(k)(z_k - z_0(k) - 1) \geq 0, \quad k = 0, 1, \dots, N. \quad (103)$$

类似地, 对于推力上界约束中的指数项 $T_{\max} e^{-z}$, 也进行相同的一阶线性化处理:

$$\sigma \leq T_{\max} e^{-z_0} [1 - (z - z_0)], \quad (104)$$

$$\sigma + T_{\max} e^{-z_0}(z - z_0 - 1) \leq 0. \quad (105)$$

定义 $\mu_2(k)$ 并得到上界线性化约束:

$$\mu_2(k) \triangleq T_{\max} e^{-z_0(k)} = \frac{\rho_2}{m_0 - \alpha \rho_2 \cdot k \Delta t}, \quad k = 0, 1, \dots, N, \quad (106)$$

$$\sigma_k + \mu_2(k)(z_k - z_0(k) - 1) \leq 0, \quad k = 0, 1, \dots, N. \quad (107)$$

3.3.4 线性化系数的物理含义与时变特性

系数 $\mu_1(k)$ 和 $\mu_2(k)$ 具有明确的物理意义: 它们分别对应最小和最大推力幅值在参考质量轨迹下产生的加速度大小。由于参考质量随燃料消耗递减, $\mu_1(k)$ 和 $\mu_2(k)$ 均随着时间步 k 的增大而单调递增。这一时变特性准确反映了燃料消耗导致飞行器质量下降、同样推力产生更大加速度的物理过程。

线性化约束 (103) 和 (107) 与二阶锥约束 $\|\mathbf{u}\| \leq \sigma$ 一起, 完整地描述了推力约束在新变量空间中的凸近似。原始的 121 维非凸约束集 (三维球体的外部区域) 被替换为 $(N+1)$ 个时变线性不等式配合 $(N+1)$ 个二阶锥约束, 整个约束系统成为凸约束。

需要指出的是, 线性化引入的近似误差在参考轨迹附近较小, 但随 $|z - z_0|$ 增大而增大。在制导算法的实际实现中, 通过迭代求解与参考轨迹更新 (即 Successive Convexification 框架) 来保证线性化的精度。若当前迭代解 z_k 与 $z_0(k)$ 偏差过大, 则更新 $z_0(k) \leftarrow z_k$ 并重新求解 SOCP, 直至收敛。

3.4 下滑角约束的凸性分析

下滑角约束是确保飞行器沿安全空间走廊接近着陆点的几何约束, 其数学形式为:

$$\|[r_y(t), r_z(t)]^T\| \leq r_x(t) \tan \theta. \quad (108)$$

该约束在位置空间 (r_x, r_y, r_z) 中定义了一个以 r_x 正半轴为中心线、半顶角为 θ 的圆锥形可行区域。从凸分析的角度看, 该约束可直接改写为如下标准二阶锥形式:

$$[r_x(t) \tan \theta, r_y(t), r_z(t)]^T \in \mathcal{K}_3. \quad (109)$$

由于二阶锥 $\mathcal{K}_3$ 是凸锥, 上述约束是凸约束, 无需任何额外的松弛或线性化处理。在离散化后, 只需在每个离散节点 t_k 处施加一个 3 维二阶锥约束即可保持问题的凸性。

这里需要补充说明一个关键细节: 为保证 $r_x(t) \tan \theta$ 非负 (二阶锥的第一个分量必须非负), 要求 $r_x(t) \geq 0$ 始终成立。在着陆制导问题中, r_x 定义为飞行器距着陆点的前向距离, 在有效飞行阶段始终为正, 因此该条件自然满足。

3.5 直接转录法离散化

将连续时间最优控制问题转化为有限维凸优化问题, 采用直接转录法 (Direct Transcription) 进行时间离散化。直接转录法的核心思想是将无限维的控制变量和状态变量在有限个时间节点上离散化, 并将连续动力学方程转化为代数约束。

3.5.1 时间网格划分

将总飞行时间 t_f 均匀划分为 $N = 30$ 个区间, 每个区间长度:

$$\Delta t = \frac{t_f}{N}. \quad (110)$$

对于 $t_f = 81$ s, 得到 $\Delta t = 2.7$ s。这一定时间间隔的选择基于以下考虑: 一方面需要足够精细以准确近似连续动力学 (尤其是软着陆阶段的速度和位置变化), 另一方面需控制问题规模以保证实时求解的计算效率。 $N = 30$ 的取值经过了数值实验验证, 在精度和效率之间取得了良好的平衡。

离散时间节点记为:

$$t_k = k\Delta t, \quad k = 0, 1, \dots, N. \quad (111)$$

在节点 t_k 处的状态和控制分别记为:

$$\mathbf{r}_k \triangleq \mathbf{r}(t_k), \quad \mathbf{v}_k \triangleq \mathbf{v}(t_k), \quad z_k \triangleq z(t_k), \quad (112)$$

$$\mathbf{u}_k \triangleq \mathbf{u}(t_k), \quad \sigma_k \triangleq \sigma(t_k). \quad (113)$$

3.5.2 双积分器离散化

位置动力学 $\dot{\mathbf{r}} = \mathbf{v}$ 和速度动力学 $\dot{\mathbf{v}} = \mathbf{g} + \mathbf{u}$ 构成双积分器系统。在区间 $[t_k, t_{k+1}]$ 上, 对速度方程采用前向欧拉离散, 再代入位置方程得双积分器格式:

$$\mathbf{r}_{k+1} = \mathbf{r}_k + \Delta t \cdot \mathbf{v}_k + \frac{1}{2}\Delta t^2(\mathbf{g} + \mathbf{u}_k), \quad (114)$$

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \Delta t \cdot (\mathbf{g} + \mathbf{u}_k), \quad (115)$$

$$z_{k+1} = z_k - \Delta t \cdot \alpha \sigma_k, \quad (116)$$

其中 $k = 0, \dots, N-1$, $\Delta t = t_f/N = 81/30 = 2.7$ s。重力向量 $\mathbf{g} = [-g, 0, 0]^\top$ ($g = 3.7114$ m/s², 仅作用于 x 方向)。

展开为 7 个标量方程 (与 C 代码 MarsLanding.c 第 246–290 行完全一致):

$$r_{x,k+1} = r_{x,k} + \Delta t \cdot v_{x,k} + \frac{1}{2}\Delta t^2 \cdot u_{x,k} - \frac{1}{2}g \Delta t^2, \quad (117)$$

$$r_{y,k+1} = r_{y,k} + \Delta t \cdot v_{y,k} + \frac{1}{2}\Delta t^2 \cdot u_{y,k}, \quad (118)$$

$$r_{z,k+1} = r_{z,k} + \Delta t \cdot v_{z,k} + \frac{1}{2}\Delta t^2 \cdot u_{z,k}, \quad (119)$$

$$v_{x,k+1} = v_{x,k} + \Delta t \cdot u_{x,k} - g \Delta t, \quad (120)$$

$$v_{y,k+1} = v_{y,k} + \Delta t \cdot u_{y,k}, \quad (121)$$

$$v_{z,k+1} = v_{z,k} + \Delta t \cdot u_{z,k}, \quad (122)$$

$$z_{k+1} = z_k - \Delta t \cdot \alpha \sigma_k. \quad (123)$$

在 ECOS 的 $Ax = b$ 标准形式中, 方程(117)写为等式约束 $+r_{x,k} + \Delta t v_{x,k} + \frac{1}{2}\Delta t^2 u_{x,k} - r_{x,k+1} = +\frac{1}{2}g\Delta t^2$ ——重力项以正值出现在 b 向量右侧, 即 C 代码中 CRS_ROW($g * 0.5 * dt * dt$)。这是理解 C 手写矩阵构造中 b 向量符号的关键。

3.5.3 边界条件的离散化

初始条件在 $k = 0$ 节点处施加:

$$\mathbf{r}_0 = \mathbf{r}_0, \quad \mathbf{v}_0 = \mathbf{v}_0, \quad z_0 = \ln m_0. \quad (124)$$

终端条件在 $k = N$ 节点处施加 (软着陆要求速度为零):

$$\mathbf{r}_N = \mathbf{r}_f, \quad \mathbf{v}_N = \mathbf{v}_f. \quad (125)$$

3.5.4 推力约束与下滑角约束的离散化

推力约束在每个离散节点 k 处离散化如下:

- 二阶锥约束 (推力松弛, 共 $N + 1 = 31$ 个):

$$\|\mathbf{u}_k\| \leq \sigma_k, \quad [\sigma_k, u_{x,k}, u_{y,k}, u_{z,k}]^T \in \mathcal{K}_4. \quad (126)$$

- 线性化推力下界约束 (共 $N + 1 = 31$ 个线性不等式):

$$\sigma_k + \mu_1(k)(z_k - z_0(k) - 1) \geq 0. \quad (127)$$

- 线性化推力上界约束 (共 $N + 1 = 31$ 个线性不等式):

$$\sigma_k + \mu_2(k)(z_k - z_0(k) - 1) \leq 0. \quad (128)$$

下滑角约束同样在每个离散节点 k 处施加 ($k = 0, 1, \dots, N$):

$$\|[r_{y,k}, r_{z,k}]^T\| \leq r_{x,k} \tan \theta, \quad [r_{x,k} \tan \theta, r_{y,k}, r_{z,k}]^T \in \mathcal{K}_3. \quad (129)$$

3.5.5 质量边界约束与目标函数的离散化

质量对数的物理边界对应于结构干质量和初始湿质量:

$$\ln m_{\min} \leq z_k \leq \ln m_{\max}, \quad k = 0, 1, \dots, N, \quad (130)$$

其中 $m_{\min} = m_{\text{dry}}$ 为干质量 (不含推进剂的飞行器结构质量), $m_{\max} = m_0$ 为初始质量。

燃料最优目标 $J = -m(t_f) = -e^{z(t_f)}$ 在离散形式中利用指数函数的单调性简化为直接最小化 $-z_N$:

$$\min J = -z_N. \quad (131)$$

这是因为 e^z 是 z 的严格单调递增函数, 最小化 $-e^{z_N}$ 等价于最小化 $-z_N$, 而后者是变量的线性函数, 保持了问题的凸性。

3.6 离散 SOCP 问题的完整形式

综合上述所有离散化约束, 完整的离散 SOCP 问题 $\mathcal{P}_{\text{SOCP}}$ 可表述为如下标准形式的凸优化问题。

3.6.1 决策变量与维度分析

问题的决策变量向量将所有节点上的状态变量和控制变量按节点顺序排列:

$$\mathbf{x} = [\mathbf{r}_0, \dots, \mathbf{r}_N, \mathbf{v}_0, \dots, \mathbf{v}_N, z_0, \dots, z_N, \quad (132)$$

$$\mathbf{u}_0, \dots, \mathbf{u}_N, \sigma_0, \dots, \sigma_N]^\top \in \mathbb{R}^{341}. \quad (133)$$

变量总数为 $(N+1) \times (3+3+1+3+1) = 31 \times 11 = 341$, 其构成如下:

- 位置变量: $3 \times (N+1) = 93$ 个 ($\mathbf{r}_k \in \mathbb{R}^3$);
- 速度变量: $3 \times (N+1) = 93$ 个 ($\mathbf{v}_k \in \mathbb{R}^3$);
- 质量对数变量: $1 \times (N+1) = 31$ 个 ($z_k \in \mathbb{R}$);
- 加速度控制变量: $3 \times (N+1) = 93$ 个 ($\mathbf{u}_k \in \mathbb{R}^3$);
- 推力松弛变量: $1 \times (N+1) = 31$ 个 ($\sigma_k \in \mathbb{R}$)。

3.6.2 目标函数

最小化燃料消耗的等价形式为:

$$\min_{\mathbf{x}} J = -z_N. \quad (134)$$

该目标函数是决策变量的线性函数, 为 SOCP 提供了标准形式的线性目标。

3.6.3 等式约束系统

等式约束共计 223 行, 包含以下三类:

1. 初始条件约束 (7 行):

$$\mathbf{r}_0 = \mathbf{r}_0, \quad \mathbf{v}_0 = \mathbf{v}_0, \quad z_0 = \ln m_0. \quad (135)$$

2. 终端条件约束 (6 行):

$$\mathbf{r}_N = \mathbf{r}_f, \quad \mathbf{v}_N = \mathbf{v}_f. \quad (136)$$

3. 动力学递推约束 ($7N = 210$ 行, $k = 0, \dots, N-1$):

$$\mathbf{r}_{k+1} - \mathbf{r}_k - \Delta t \cdot \mathbf{v}_k = \mathbf{0}, \quad (137)$$

$$\mathbf{v}_{k+1} - \mathbf{v}_k - \Delta t \cdot (\mathbf{g} + \mathbf{u}_k) = \mathbf{0}, \quad (138)$$

$$z_{k+1} - z_k + \Delta t \cdot \alpha \sigma_k = 0. \quad (139)$$

等式约束总数验证: $7 + 6 + 210 = 223$ 。

3.6.4 不等式约束系统

不等式约束共计 341 行，按以下四类分布：

1. **质量值约束** (62 行)：线性化的推力上下界约束。这类约束包含了质量对数变量 z_k 和推力松弛变量 σ_k 的耦合关系，代表了对原始推力幅值约束的线性化近似。

$$\sigma_k + \mu_1(k)(z_k - z_0(k) - 1) \geq 0, \quad k = 0, \dots, N, \quad (140)$$

$$\sigma_k + \mu_2(k)(z_k - z_0(k) - 1) \leq 0, \quad k = 0, \dots, N. \quad (141)$$

共 $2(N + 1) = 62$ 行。

2. **质量边界约束** (62 行)：质量对数变量的简单上下界。

$$\ln m_{\min} \leq z_k \leq \ln m_{\max}, \quad k = 0, \dots, N. \quad (142)$$

共 $2(N + 1) = 62$ 行。

3. **下滑角锥约束** (93 行)：将 31 个 3 维二阶锥约束展开为标量不等式。

$$r_{x,k} \tan \theta \geq \sqrt{r_{y,k}^2 + r_{z,k}^2}, \quad k = 0, \dots, N. \quad (143)$$

共 $N + 1 = 31$ 个锥约束，展开为 $3(N + 1) = 93$ 行标量不等式。

4. **推力锥约束** (124 行)：将 31 个 4 维二阶锥约束展开为标量不等式。

$$\sigma_k \geq \sqrt{u_{x,k}^2 + u_{y,k}^2 + u_{z,k}^2}, \quad k = 0, \dots, N. \quad (144)$$

共 $N + 1 = 31$ 个锥约束，展开为 $4(N + 1) = 124$ 行标量不等式。

不等式约束总数验证： $62 + 62 + 93 + 124 = 341$ 。有趣的是，不等式约束的行数与决策变量的维数恰好相等（均为 341），这是一种巧合而非必然。

3.6.5 锥约束的标准二阶锥形式

将不等式约束中的旋转锥约束归纳为 62 个标准的二阶锥约束，以便直接输入 SOCP 求解器：

$$[r_{x,k} \tan \theta, r_{y,k}, r_{z,k}]^T \in \mathcal{K}_3, \quad k = 0, \dots, N, \quad (145)$$

$$[\sigma_k, u_{x,k}, u_{y,k}, u_{z,k}]^T \in \mathcal{K}_4, \quad k = 0, \dots, N. \quad (146)$$

其中包含 31 个 3 维锥（下滑角约束）和 31 个 4 维锥（推力松弛约束），共 62 个二阶锥约束。

3.6.6 问题规模汇总与算法选择

表 2 汇总了离散 SOCP 问题的完整规模统计。

表 2: 离散 SOCP 问题规模统计

类别	数量	说明
决策变量	341	31 节点 $\times$ 11 维
等式约束	223	7 初始 + 6 终端 + 210 动力学
不等式约束 (标量行)	341	62 质量值 + 62 质量边界 + 93 下滑角 + 124 推力
二阶锥约束	62	31 个 $\mathcal{K}_3$ + 31 个 $\mathcal{K}_4$

该问题是一个中等规模的标准凸二阶锥规划, 具有如下结构特点: (1) 等式约束矩阵呈带状稀疏结构, 每个动力学递推方程仅涉及相邻两个时间节点的变量; (2) 锥约束在每节点解耦, 无跨节点耦合; (3) 目标函数极为简单 (单变量线性函数)。这些结构特点使得问题非常适合采用原始-对偶内点法求解。业界成熟的 SOCP 求解器如 ECOS、GuRoBi、MOSEK 等均可在毫秒至秒级时间内稳定求解该问题。

3.7 Lossless Convexification 的理论保证

上述凸化过程中, 一个关键步骤是将等式约束 $\sigma = \|\mathbf{u}\|$ 松弛为不等式 $\sigma \geq \|\mathbf{u}\|$ 。这一松弛扩大了问题的可行域——在物理上, 允许 $\sigma > \|\mathbf{u}\|$ 意味着推力-质量比 $\|\mathbf{T}\|/m$ 大于实际加速度 $\|\mathbf{u}\|$, 对应于一种“非物理的推力放大”状态。如果 SOCP 的最优解恰好处于这种松弛区域 (即 $\sigma^* > \|\mathbf{u}^*\|$), 则问题的解不再具有物理意义。Lossless Convexification 理论 [8] 保证了在满足一定条件下, 这种情况不会发生, 从而松弛是“无损的”。

3.7.1 核心定理陈述

Açikmeşe 等人 (2013) 在文献 [11] 中证明了以下核心定理:

定理 3.1 (Lossless Convexification). 考虑由动力学方程 (86)–(88)、推力约束 $\|\mathbf{u}\| \leq \sigma$ 和 $\sigma \geq T_{\min}e^{-z}$ (经线性化近似后)、以及边界条件构成的最优控制问题。如果存在可行解且问题参数满足:

$$T_{\max} > T_{\min} > 0, \quad m_0 > m_{\min} > 0, \quad (147)$$

则松弛后 SOCP 问题的最优解 $(\mathbf{u}^*, \sigma^*)$ 必然满足 $\sigma^* = \|\mathbf{u}^*\|$ 对几乎所有 $t \in [0, t_f]$ 成立。

3.7.2 证明思路与能量论证

该定理的完整证明涉及庞特里亚金极小值原理和最优控制理论的深入分析, 此处给出证明的核心思路和关键步骤:

1. **能量消耗视角:** 燃料最优目标等价于最小化总推力冲量 $\int \|\mathbf{T}\| dt$ 。在变量变换后的系统中, 燃料消耗率由 $-\dot{z} = \alpha\sigma$ 决定。如果 $\sigma > \|\mathbf{u}\|$, 则该时刻的燃料消耗率高于产生当前加速度所需的水平, 是无谓的浪费。因此, 在燃料最优解中, 理性的控制策略必然使 $\sigma = \|\mathbf{u}\|$ 。

2. **庞特里亚金极小值原理**：应用极小值原理于松弛后的 SOCP 问题，构造 Hamiltonian 函数 H ，并分析最优控制必须满足的条件。通过协态方程和横截条件可以证明，与松弛约束 $\sigma \geq \|\mathbf{u}\|$ 对应的 Lagrange 乘子在最优解处必须非负，且互补松弛条件迫使 $\sigma^* = \|\mathbf{u}^*\|$ 几乎处处成立。
3. **严格数学证明的技术细节**：Açikmeşe 等人 (2013) 的证明通过以下步骤完成：(a) 将原非凸问题转化为“松弛问题” (Relaxed Problem, RP)；(b) 证明 RP 是一个凸优化问题；(c) 通过构造 RP 的 KKT 条件，证明 RP 的任意最优解必然满足 $\sigma^* = \|\mathbf{u}^*\|$ ；(d) 由于 $\sigma^* = \|\mathbf{u}^*\|$ 意味着松弛是紧的，因此 RP 的解也是原非凸问题的可行解；(e) 通过反证法证明 RP 的最优值就是原问题的最优值。
4. **凸性的关键作用**：由于 SOCP 是凸优化问题 [12]，任何局部最优解都是全局最优解，这确保了求解器找到的解就是全局最优解。若原非凸问题存在多个局部最优解，则 SOCP 的全局最优解对应于其中的最优者。

3.7.3 直观物理解释与工程意义

从物理角度理解，Lossless Convexification 的直观解释是：推力松弛变量 σ 在目标函数中受到隐性惩罚。因为 σ 越大，质量对数 z 的衰减越快 ($\dot{z} = -\alpha\sigma$)，从而导致终端质量 $m(t_f) = e^{z(t_f)}$ 减小，与燃料最优目标（最大化 $m(t_f)$ ）直接矛盾。因此，最优解自然倾向于使 σ 尽可能小，直到下界 $\sigma \geq \|\mathbf{u}\|$ 取等号。这一机制使得松弛约束在最优解处自然地“紧”起来，保证了凸化过程的无损性。

3.7.4 在本文问题中的适用性论证

本文的着陆制导问题满足定理的全部前提条件：

- $T_{\max} = 3100 \text{ N} > T_{\min} = 930 \text{ N} > 0$ ，满足推力幅值的严格大小关系；
- $m_0 = 1905 \text{ kg} > m_{\min} = 1505 \text{ kg} > 0$ ，存在可供消耗的推进剂；
- 推力下界约束经过一阶泰勒展开线性化后，在参考轨迹附近保持了对原约束的局部近似，且可通过序列迭代修正线性化误差。

因此，可以严格保证 SOCP 求解结果与原非凸最优控制问题的解在工程允许精度范围内一致。线性化引入的近似误差可通过序列迭代 (Successive Convexification) 框架进行补偿：在当前迭代解 $\{z_k\}$ 的基础上更新参考轨迹 $z_0(k)$ ，并重新线性化、重新求解 SOCP，直至相邻两次迭代的解满足收敛判据（如 $\max_k |z_k - z_0(k)| < \varepsilon$ ）。该迭代过程的详细实现在算法章节（第 4 章）中讨论。

3.8 本章小结

通过上述无损凸化和离散化处理，本章完成了一个完整的理论推导链条：从非凸最优控制问题出发，通过变量变换消除动力学非线性，通过二阶锥松弛和泰勒展开线性化处理非凸推力约束，通过直接转录法实现时间离散化，最终将问题转化为标准形式的有限维二阶锥规划。最关

键的是, Lossless Convexification 理论 [8] 保证了松弛过程的精确性, 使得 SOCP 的解与原非凸问题的最优解一致。第 4 章将在此基础上设计完整的迭代制导算法, 第 5 章通过数值仿真验证该理论框架的有效性和精度。

4 稀疏矩阵构造与 ECOS 求解器

4.1 ECOS 求解器概述

ECOS (Embedded Conic Solver) 是由 embotech 公司开发的开源二阶锥规划求解器 [13], 专为嵌入式实时优化场景设计 [14]。其核心算法基于 Mehrotra 预测-校正原对偶内点法 (Mehrotra-type Predictor-Corrector Primal-Dual Interior Point Method), 该算法在每次迭代中通过求解两组线性系统——先计算仿射方向 (预测步), 再利用该方向信息动态调整中心化参数并计算校正方向——显著减少了达到最优解所需的迭代次数。

ECOS 接受如下标准形式的锥优化问题:

$$\min_{\mathbf{x} \in \mathbb{R}^n} \quad \mathbf{c}^\top \mathbf{x} \quad (148)$$

$$\text{s.t.} \quad \mathbf{A}\mathbf{x} = \mathbf{b}, \quad (149)$$

$$\mathbf{h} - \mathbf{G}\mathbf{x} \in \mathcal{K}, \quad (150)$$

其中 $\mathcal{K}$ 为多个锥的笛卡尔积:

$$\mathcal{K} = \mathbb{R}_+^l \times \mathcal{K}_{q_1} \times \mathcal{K}_{q_2} \times \cdots \times \mathcal{K}_{q_m}. \quad (151)$$

具体而言, $\mathcal{K}$ 的前 l 个维度对应非负象限 (即线性不等式约束, ECOS 的参数 l 即为线性约束行数), 后续各段为不同维度的二阶锥。二阶锥 $\mathcal{K}_q$ 的定义为

$$\mathcal{K}_q = \left\{ (x_0, x_1, \dots, x_{q-1}) \in \mathbb{R}^q : x_0 \geq \sqrt{x_1^2 + \cdots + x_{q-1}^2} \right\}. \quad (152)$$

ECOS 在 NASA 火星着陆 MSL 任务的后备制导系统 FNPEG 中得到工程验证, 并在机器人运动规划、MPC 控制、信号处理等嵌入式场景中广泛部署。求解器以纯 C 语言实现, 核心代码仅约 8000 行, 无外部线性代数库依赖, 通过 LDL 稀疏分解 [15] 求解内点法所需的 KKT 系统。求解器核心参数如表 3 所示。

表 3: ECOS 核心求解参数

参数	含义	默认值
MAXIT	最大迭代次数	100
FEASTOL	原始/对偶不可行容差	1×10^{-8}
ABSTOL	对偶间隙绝对容差	1×10^{-8}
RELTOL	对偶间隙相对容差	1×10^{-8}
DELTASTAT	静态正则化参数	7×10^{-8}
DELTA	动态正则化参数	2×10^{-7}
EPS	正则化阈值	1×10^{-13}
NITREF	迭代精化步数	9
GAMMA	最终步长缩放因子	0.99
EQUIL_ITERS	Ruiz 均衡化迭代轮数	3

值得特别指出的是, ECOS 采用 Ruiz 均衡化 (Ruiz Equilibration) 算法对问题数据进行预处理。该算法通过 $\sqrt{\|\cdot\|_\infty/\|\cdot\|_1}$ 的缩放比例对矩阵行和列进行交替缩放, 经过 3 轮迭代可将原问题的条件数显著压缩。对于本文的火星着陆问题, 均衡化前的 KKT 矩阵条件数约为 $10^{4.9}$ (动态范围来自位置量 $r \sim 10^3$ 、质量对数 $z \sim \mathcal{O}(1)$ 、推力加速度 $u \sim 10^{-3}$ 三者的量级差异), 经 3 轮 Ruiz 均衡化后降至约 4.1, 极大改善了对偶内点法的数值稳定性。均衡化后, ECOS 调用 AMD (Approximate Minimum Degree) 算法对 KKT 矩阵列进行重排序, 通过贪心策略选择度数最小的节点作为消去元的下一列, 可有效减少 LDL 分解过程中的填充元数量, 进而降低分解的计算复杂度和内存占用。对于 $n = 341$ 的 KKT 矩阵, AMD 排序可将 LDL 分解的浮点运算量减少约 40%–60%。

LDL 分解过程同时应用了静态正则化和动态正则化两种机制。静态正则化 (参数 $\delta_{\text{stat}} = 7 \times 10^{-8}$) 在每次分解前向 KKT 矩阵对角线加入固定小量; 动态正则化 (参数 $\delta_{\text{dyn}} = 2 \times 10^{-7}$) 则在搜索方向计算不可靠时被激活, 逐步增大正则化量直至分解稳定。两种正则化机制的协同作用保证了 KKT 系统在病态条件下的可解性。每次 LDL 求解后, ECOS 执行最多 9 步迭代精化 (Iterative Refinement), 每次精化将 LDL 分解的残差代入再求解, 理论上可将相对残差降低至 $\epsilon_{\text{mach}}^{N_{\text{IR}}}$ 量级, 将残差降低约 10^6 倍, 进一步补偿因有限精度计算导致的精度损失。

4.2 ECOS 接口标准与符号约定

ECOS 的锥约束接口将不等式约束矩阵 $\mathbf{G}$ 的 m 行划分为两段: 前 l 行为线性约束, 后 $m-l$ 行为锥约束, 二者不可交错排列。线性约束段解释为 $\mathbf{G}[i, :]\mathbf{x} \leq \mathbf{h}[i]$ (等价于 $\mathbf{h} - \mathbf{G}\mathbf{x}$ 的第 i 个分量位于非负象限), 而锥约束段解释为 $\mathbf{h}_j - \mathbf{G}_j\mathbf{x} \in \mathcal{K}_{q_j}$, 即第 j 个锥的 $\mathbf{h} - \mathbf{G}\mathbf{x}$ 向量片段位于 q_j 维二阶锥内。

这一符号约定是 ECOS 接口中最关键也最容易出错的环节。许多初学者直观地将二阶锥约束 $\|\mathbf{x}_1\| \leq t$ 理解为 $\mathbf{G}\mathbf{x} \leq \mathbf{h}$ 的形式, 直接写出 $\mathbf{G}\mathbf{x} = [t, x_1, x_2, \dots]^\top$ 和 $\mathbf{h} = \mathbf{0}$ 。这对应于 $\mathbf{h} - \mathbf{G}\mathbf{x} = -[t, x_1, x_2, \dots]^\top$, 即 $\mathbf{h} - \mathbf{G}\mathbf{x}$ 的第一个分量是 $-t$ 而非 t , 代入二阶锥定义得到 $-t \geq \sqrt{x_1^2 + \dots}$, 这等价于 $t \leq 0$, 完全颠倒了约束的物理含义。

正确的做法是令 $\mathbf{G}$ 采用负单位矩阵, 使得 $\mathbf{h} - \mathbf{G}\mathbf{x} = \mathbf{0} - (-\mathbf{I})\mathbf{x} = \mathbf{x}$, 从而 $\mathbf{h} - \mathbf{G}\mathbf{x}$ 的第一个分量直接等于 t 。在本项目的手写矩阵构造中, 这一约定通过 `G_PUSH(col, -val)` 宏实现: 推送非零元时系数取负, 确保 $\mathbf{h} - \mathbf{G}\mathbf{x}$ 各分量符号正确。

CasADi 自动微分工具提取 ECOS 矩阵时遵循类似的公式。定义 CasADi 符号约束向量 $\mathbf{eq}(\mathbf{x}) = \mathbf{0}$ (等式约束) 和 $\mathbf{ineq}(\mathbf{x}) \leq \mathbf{0}$ (不等式约束), 则 ECOS 矩阵的提取公式为

$$\mathbf{A} = \nabla_{\mathbf{x}} \mathbf{eq}(\mathbf{0}), \quad \mathbf{b} = -\mathbf{eq}(\mathbf{0}), \quad (153)$$

$$\mathbf{G} = \nabla_{\mathbf{x}} \mathbf{ineq}(\mathbf{0}), \quad \mathbf{h} = -\mathbf{ineq}(\mathbf{0}). \quad (154)$$

验算上述公式的正确性可从 $\mathbf{A}\mathbf{x} = \mathbf{b}$ 出发: 将 $\mathbf{eq}(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{eq}(\mathbf{0})$ (因 $\mathbf{eq}$ 为线性函数) 代入 $\mathbf{eq}(\mathbf{x}) = \mathbf{0}$ 得 $\mathbf{A}\mathbf{x} = -\mathbf{eq}(\mathbf{0})$, 故 $\mathbf{b} = -\mathbf{eq}(\mathbf{0})$ 。对于不等式, $\mathbf{ineq}(\mathbf{x}) = -\mathbf{G}\mathbf{x} + \mathbf{ineq}(\mathbf{0})$, 代入 $\mathbf{ineq}(\mathbf{x}) \leq \mathbf{0}$ 得 $\mathbf{h} - \mathbf{G}\mathbf{x} \geq \mathbf{0}$, 其中 $\mathbf{h} = -\mathbf{ineq}(\mathbf{0})$ 。

4.3 稀疏矩阵存储格式

4.3.1 压缩行存储 CRS

CRS (Compressed Row Storage) 格式按行存储稀疏矩阵。设矩阵有 m 行、 n 列、 n_{nz} 个非零元, CRS 使用三个数组存储:

- `CRpr[0..nnz-1]`: 按行优先顺序存储非零元的数值;
- `CRjc[0..nnz-1]`: 存储每个非零元的列号 (0-based);
- `CRir[0..m]`: 行指针数组, `CRir[i]` 为第 i 行第一个非零元在 `CRpr/CRjc` 中的索引, 第 i 行的非零元个数为 `CRir[i+1] - CRir[i]`。

CRS 格式的优势在于按行构造极其直观——逐行遍历约束条件, 每遇到一个非零元即调用 `CRS_PUSH(col, val)` 推入列号和数值, 每完成一行约束调用 `CRS_ROW(b_val)` 记录行指针并填入 $\mathbf{b}$ 向量值。这种构造方式与数学建模的思维流程完全一致: 先逐条枚举约束方程, 再逐项写入对应系数。

4.3.2 压缩列存储 CCS

CCS (Compressed Column Storage) 格式按列存储, 是 CRS 的对称形式。设矩阵有 m 行、 n 列、 n_{nz} 个非零元:

- `CCpr[0..nnz-1]`: 按列优先顺序存储非零元数值;
- `CCir[0..nnz-1]`: 存储每个非零元的行号;
- `CCjc[0..n]`: 列指针数组, `CCjc[j]` 为第 j 列第一个非零元在 `CCpr/CCir` 中的索引。

ECOS 要求输入的 $\mathbf{A}$ 和 $\mathbf{G}$ 矩阵均为 CCS 格式, 因此必须先以 CRS 格式构造完毕, 再转换为 CCS。在 C 代码实现中, `CRS_PUSH` 和 `CRS_ROW` 被封装为宏而非函数, 原因在于宏调用时编

译器可直接展开为数组赋值操作, 避免了函数调用开销——对于本文 $733 + 403 = 1136$ 次非零元推送的总调用次数而言, 宏展开可节省约数十微秒的构造时间。CRS 格式的另一个优点在于构造阶段无需预先知道每行的非零元个数: CRS_ROW 宏通过 $\text{CRAir}[\text{idx_row}+1] = \text{CRAir}[\text{idx_row}] + \text{row_nnz}$ 自动将当前行计数写入行指针数组, 完全消除了对动态内存分配的需求, 使矩阵构造可在栈上定长数组内完成。

4.3.3 CRS 到 CCS 的转换算法

CRS 到 CCS 的转换可以通过四遍线性扫描完成, 时间复杂度 $\mathcal{O}(n_{nz})$, 额外空间 $\mathcal{O}(n)$ 。算法 1 给出了具体步骤。

Algorithm 1 CRS 到 CCS 格式转换

Require: CRS 格式数组 CRjc, CRir, CRpr; 矩阵行数 m 、列数 n 、非零元总数 n_{nz}

Ensure: CCS 格式数组 CCjc, CCir, CCpr

```

1: 分配工作数组  $w[0..n-1]$  ▷ 第一遍: 统计每列非零元个数
2: for  $j = 0$  to  $n - 1$  do
3:    $w[j] \leftarrow 0$ 
4: end for
5: for  $k = 0$  to  $n_{nz} - 1$  do
6:    $w[\text{CRjc}[k]] \leftarrow w[\text{CRjc}[k]] + 1$ 
7: end for
▷ 第二遍: 前缀和得到 CCS 列指针
8:  $\text{CCjc}[0] \leftarrow 0$ 
9: for  $j = 0$  to  $n - 1$  do
10:   $\text{CCjc}[j+1] \leftarrow \text{CCjc}[j] + w[j]$ 
11: end for
▷ 第三遍: 重置工作数组为每列写入位置起点
12: for  $j = 0$  to  $n - 1$  do
13:   $w[j] \leftarrow \text{CCjc}[j]$ 
14: end for
▷ 第四遍: 按行遍历 CRS, 填入 CCS
15: for  $i = 0$  to  $m - 1$  do
16:   for  $k = \text{CRir}[i]$  to  $\text{CRir}[i+1] - 1$  do
17:     $\text{pos} \leftarrow w[\text{CRjc}[k]]$ 
18:     $w[\text{CRjc}[k]] \leftarrow w[\text{CRjc}[k]] + 1$ 
19:     $\text{CCir}[\text{pos}] \leftarrow i$ 
20:     $\text{CCpr}[\text{pos}] \leftarrow \text{CRpr}[k]$ 
21:   end for
22: end for

```

算法的核心思想是，CRS 和 CCS 共享相同的非零元集合，仅按行列优先级不同排序。第一遍扫描统计每列的非零元个数，第二遍通过前缀和得到 CCS 列指针，第三遍将工作数组重置为每列当前的写入位置，第四遍按 CRS 的行优先序遍历将每个非零元填入 CCS 的正确位置。由于 CRS 的行优先顺序保证了非零元在每行内部按列递增排列，填入 CCS 时每列内部自然保持行号递增，满足 CCS 的排序要求。该算法无需对非零元进行排序操作，完全避免了比较和交换的开销——这是其达到 $\mathcal{O}(n_{nz})$ 线性时间复杂度的关键。

4.4 矩阵构造详解

本文的手写稀疏矩阵构造摒弃了通用建模层（如 CVXPY 或 YALMIP），以纯 C 语言直接向 CRS 格式填入非零元。优化变量 $\mathbf{x} \in \mathbb{R}^{341}$ 按时间步排列，每步 11 维：

$$\mathbf{x}_k = [r_{x,k}, r_{y,k}, r_{z,k}, v_{x,k}, v_{y,k}, v_{z,k}, z_k, u_{x,k}, u_{y,k}, u_{z,k}, \sigma_k]^\top, \quad (155)$$

其中 $z_k = \ln m_k$ 为质量对数， u_k 为推力加速度， σ_k 为推力松弛变量。

4.4.1 等式约束矩阵 $\mathbf{A}$ (223×341)

等式约束矩阵 $\mathbf{A}$ 包含 733 个非零元，由三部分构成。

边界条件生成 13 个非零元：前 7 行固定初始状态 $\mathbf{x}_0$ （位置 $\mathbf{r}_0$ 、速度 $\mathbf{v}_0$ 、质量对数 z_0 ），每行 1 个非零元（对角线元素 1）；后 6 行固定终端状态 $\mathbf{x}_N$ 的位置和速度，质量不受约束，因此仅 6 行而非 7 行。边界条件共计 $7 + 6 = 13$ 个非零元。

动力学约束生成 720 个非零元，每步 24 个非零元 $\times 30$ 步。三个位置方向 (r_x, r_y, r_z) 各需 4 个非零元 (r_k, v_k, u_k, r_{k+1}) ，三个速度方向 (v_x, v_y, v_z) 各需 3 个非零元 (v_k, u_k, v_{k+1}) ，质量对数方程需 3 个非零元 (z_k, σ_k, z_{k+1}) 。合计 $12 + 9 + 3 = 24$ 个非零元/步。

以 r_x 的离散动力学为例，梯形积分格式为

$$r_{x,k+1} = r_{x,k} + \Delta t \cdot v_{x,k} + \frac{1}{2}\Delta t^2 \cdot u_{x,k} - \frac{1}{2}g\Delta t^2, \quad (156)$$

对应等式约束 $\mathbf{Ax} = \mathbf{b}$ 中的行：

$$\begin{bmatrix} 1 & \Delta t & \frac{1}{2}\Delta t^2 & -1 \end{bmatrix} \begin{bmatrix} r_{x,k} \\ v_{x,k} \\ u_{x,k} \\ r_{x,k+1} \end{bmatrix} = \frac{1}{2}g\Delta t^2. \quad (157)$$

表 4: 等式约束矩阵 $\mathbf{A}$ 非零元分布

约束类型	行数	每行非零元	非零元小计
初始边界条件	7	1	$7 \times 1 = 7$
终端边界条件	6	1	$6 \times 1 = 6$
位置动力学 r_x	30	4	$30 \times 4 = 120$
位置动力学 r_y	30	4	$30 \times 4 = 120$
位置动力学 r_z	30	4	$30 \times 4 = 120$
速度动力学 v_x	30	3	$30 \times 3 = 90$
速度动力学 v_y	30	3	$30 \times 3 = 90$
速度动力学 v_z	30	3	$30 \times 3 = 90$
质量对数动力学 z	30	3	$30 \times 3 = 90$
总计	223	—	733

4.4.2 不等式约束矩阵 $\mathbf{G}$ (341×341)

不等式约束矩阵 $\mathbf{G}$ 包含 403 个非零元, 由线性部分和锥部分组成。

线性部分 (186 个非零元): 前 124 行为线性不等式约束。其中, 质量值不等式 (mass value inequality) 每时间步 2 行, 每行 2 个非零元 (z_k 和 σ_k), 计 $31 \times 2 \times 2 = 124$ 个非零元。该约束对质量对数进行线性化逼近推力的上下界:

$$\mu_{1,k}(z_k - z_{0,k} - 1) + \sigma_k \leq 0, \quad (158)$$

$$-\mu_{2,k}(z_k - z_{0,k} - 1) - \sigma_k \leq 0, \quad (159)$$

其中 $\mu_{1,k} = \rho_1 e^{-z_{0,k}}$, $\mu_{2,k} = \rho_2 e^{-z_{0,k}}$ 为线性化系数。质量上下界约束每步 2 行, 每行 1 个非零元 (z_k 的系数 ± 1), 计 $31 \times 2 \times 1 = 62$ 个非零元。

锥部分 (217 个非零元): 后 217 行为二阶锥约束。下滑角约束每步 3 行 (r_x, r_y, r_z), 每行 1 个非零元, 计 $31 \times 3 = 93$ 个非零元。推力松弛约束每步 4 行 (σ, u_x, u_y, u_z), 每行 1 个非零元, 计 $31 \times 4 = 124$ 个非零元。

表 5: 不等式约束矩阵 $\mathbf{G}$ 非零元分布

约束类型	行范围	行数	每行非零元	非零元小计
质量值不等式 (线性)	0–61	62	2	124
质量上下界 (线性)	62–123	62	1	62
下滑角锥 (SOC q=3)	124–216	93	1	93
推力松弛锥 (SOC q=4)	217–340	124	1	124
总计	0–340	341	—	403

4.5 约束行序编排

G 矩阵的行序编排必须严格遵循 ECOS 接口约定：线性约束先行，锥约束后行。具体排布方案为：

线性约束段（行 0–123）：

- 行 0–61 (62 行)：质量值不等式。按时间步 $k = 0, \dots, N$ 排列，每步依次写下界和上界行。对应 ECOS 参数 $l = 124$ 中的前 62 行。
- 行 62–123 (62 行)：质量上下界。同样按 $k = 0, \dots, N$ 排列，每步依次写下界 ($-z_k$) 和上界 ($+z_k$)。

锥约束段（行 124–340）：

- 行 124–216 (93 行)：下滑角锥。每步 3 行 (r_x, r_y, r_z) ，构成 $\mathcal{K}_3$ 锥。锥维度数组 $q[0..30] = 3$ 。
- 行 217–340 (124 行)：推力松弛锥。每步 4 行 (σ, u_x, u_y, u_z) ，构成 $\mathcal{K}_4$ 锥。锥维度数组 $q[31..61] = 4$ 。

该编排方案的核心验证点在于： $\mathbf{h}$ 向量的第 124 个元素和第 217 个元素必须严格为零，因为每个二阶锥的第一行对应锥的标量分量，而下滑角和推力松弛锥的标量分量表达式分别为 $r_x \tan \theta$ 和 σ ，计算时 $\mathbf{h} - \mathbf{G}\mathbf{x} = \mathbf{x}$ （当 $\mathbf{G} = -\mathbf{I}$ 时标量分量正是 $r_x \tan \theta$ 本身， $\mathbf{h}$ 为零）。任何非零的 $\mathbf{h}$ 值都将引入非物理的偏移量，破坏锥约束的正确性。

4.6 关键符号验证

$\mathbf{b}$ 和 $\mathbf{h}$ 向量的数值提供了检验矩阵符号正确性的最直接途径。通过验证以下四个关键数值，可以在不逐个查看 1136 个非零元的条件下快速定位大部分符号错误：

1. $\mathbf{b}[0] = r_{x0} = 1500$ ：初始边界条件行将 $r_x(0)$ 固定为 1500 m。该值为正数，符合物理直觉——初始高度为正。
2. $\mathbf{b}[13] = \frac{1}{2}g\Delta t^2 = 13.528$ ：第一动力学步的 r_x 更新约束的右端项。该值为重力项 $-\frac{1}{2}g\Delta t^2$ 移项后的正值，验证了 x 轴向上、重力向下的符号约定。若该值为负，则重力方向写反。
3. $\mathbf{h}[124] = 0$ ：下滑角锥起始行的 $\mathbf{h}$ 值为零，验证锥标量分量的 $\mathbf{h} - \mathbf{G}\mathbf{x}$ 正确还原了 $r_x \tan \theta$ 。
4. $\mathbf{h}[217] = 0$ ：推力松弛锥起始行的 $\mathbf{h}$ 值为零，验证锥标量分量的 $\mathbf{h} - \mathbf{G}\mathbf{x}$ 正确还原了 σ 。

以上四点构成调试检查清单中的 L2 级（结果偏差大）快速排查项。其中 $\mathbf{b}[0]$ 和 $\mathbf{b}[13]$ 验证重力符号， $\mathbf{h}[124]$ 和 $\mathbf{h}[217]$ 验证 SOC 锥符号与行序。如果 $\mathbf{b}[0]$ 为负，则边界条件符号约定整体反转；如果 $\mathbf{b}[13]$ 为负，则动力学方程中重力项的移项方向写反；如果 $\mathbf{h}[124]$ 非零，则下滑角锥的 $\mathbf{G}$ 矩阵系数符号出错——这是最隐蔽的问题，因为非零元个数和矩阵维度均可能正确，但物理意义完全颠倒。

4.7 ECOS 内点法原理

本小节简要阐述 ECOS 内点法的核心求解机制，以便读者理解手写矩阵构造与求解器数值行为之间的关联。

KKT 系统结构：对于标准形式 (148)–(150)，引入原始变量 $\mathbf{x}$ 、对偶变量 $\mathbf{y}$ （等式约束）和 $\mathbf{s} \succeq_{\mathcal{K}} \mathbf{0}$ （不等式约束），内点法的 Karush–Kuhn–Tucker (KKT) 最优性条件可写为

$$\mathbf{G}^\top \boldsymbol{\lambda} + \mathbf{A}^\top \mathbf{y} + \mathbf{c} = \mathbf{0}, \quad (160)$$

$$\mathbf{A}\mathbf{x} = \mathbf{b}, \quad (161)$$

$$\mathbf{G}\mathbf{x} + \mathbf{s} = \mathbf{h}, \quad (162)$$

$$\mathbf{s} \circ \boldsymbol{\lambda} = \mu \mathbf{e}, \quad \mathbf{s}, \boldsymbol{\lambda} \in \mathcal{K}, \quad (163)$$

其中 $\boldsymbol{\lambda}$ 为锥对偶变量， $\circ$ 为 Jordan 代数中的锥乘积， μ 为障碍参数 (barrier parameter)， $\mathbf{e}$ 为锥单位元。式 (160) 为对偶可行性条件，式 (161) 为原始可行性条件，式 (162) 为松弛变量定义，式 (163) 为互补松弛条件——该条件经 Jordan 代数改写后保留了锥自对偶结构的对称性。将 Newton 法应用于上述非线性系统，得到每次迭代的线性化 KKT 系统。该系统的系数矩阵具有鞍点结构 (saddle-point structure)，即 2×2 块矩阵形式，其中 (1,1) 块为锥约束的 Hessian 贡献与正则化项之和，(2,2) 块为零矩阵。

Mehrotra 预测-校正算法是内点法领域最具里程碑意义的改进之一。传统的原对偶内点法每一步迭代采用固定的中心化参数 (通常 $\mu = \sigma \mu_g$ ，其中 $\sigma \in (0, 1)$ 为固定常数)，而 Mehrotra 算法通过预测步动态估计最优中心化参数。具体而言，每次迭代分为两个阶段：**预测步** (affine step) 先求解 $\mu = 0$ 时的纯 Newton 方向，即完全不考虑中心化的最速下降方向。计算该方向后，通过线搜索确定最大可行步长 α_{aff} ，并评估若沿纯仿射方向前进时对偶间隙的缩减程度 $\mu_{\text{aff}} = (1 - \alpha_{\text{aff}}) \mu_g$ 。**校正步** (corrector step) 则根据 μ_{aff} 与当前对偶间隙 μ_g 的比值动态确定中心化参数：

$$\mu_{\text{cent}} = \left(\frac{\mu_{\text{aff}}}{\mu_g} \right)^2 \mu_g, \quad (164)$$

该启发式的物理含义是：若纯仿射方向能大幅降低对偶间隙 (即 $\mu_{\text{aff}} \ll \mu_g$)，则说明当前解已接近最优解边界，算法应减少中心化 (μ_{cent} 小)，加速收敛；反之，若仿射方向推进困难 ($\mu_{\text{aff}} \approx \mu_g$)，则说明当前解距离最优解尚远，算法应增加中心化分量，使迭代轨迹保持在锥内部的中“心区”域而非过早触碰锥边界。这一自适应机制使 Mehrotra 算法的收敛速度明显优于固定中心化策略，尤其对存在多类锥约束耦合的问题收益显著。

Ruiz 均衡化：如 4.1 节所述，ECOS 在求解前对 $\mathbf{A}$ 、 $\mathbf{G}$ 、 $\mathbf{c}$ 、 $\mathbf{b}$ 、 $\mathbf{h}$ 进行 Ruiz 均衡化。对偶内点法对矩阵条件数极为敏感——KKT 系统的条件数约为原问题条件数的平方。通过 3 轮 $\sqrt{\|\cdot\|_\infty / \|\cdot\|_1}$ 缩放，原问题条件数从约 $10^{4.9}$ 降至约 4.1，使 KKT 系统求解的精度损失降低数个量级。

AMD 重排序与 LDL 分解：均衡化后，ECOS 调用 AMD 算法对 KKT 矩阵的列进行重排序，最小化 LDL 分解中填充元的数量。LDL 分解是稀疏对称不定矩阵分解的标准方法，对于 $n = 341$ 规模的 KKT 系统，分解时间约占总求解时间的 60%–70%。由于 LDL 分解的运算模式以不规则的内存访存 (CCS 格式的 gather/scatter) 为主，计算受内存带宽瓶颈限制，因此

SIMD 向量化指令（如 AVX/FMA）对此场景的加速效果极为有限——实测标量版与 AVX 编译版性能差异在 3% 以内，这印证了稀疏线性代数中“宽优于计算的”经典结论。

正则化与迭代精化：静态正则化参数 $\delta_{\text{stat}} = 7 \times 10^{-8}$ 在每次 LDL 分解前加入 KKT 矩阵对角线，避免因零对角元导致的分解失败。当搜索方向计算被标记为不可靠时，ECOS 逐步增大正则化系数直至分解稳定，最大可增至原正则化量的数百倍。每次 LDL 求解后执行 $N_{\text{IR}} = 9$ 步迭代精化，理论上可将残差降低至机器精度的 $\epsilon_{\text{mach}}^{N_{\text{IR}}}$ 量级，实际受问题条件数限制通常达到 10^{-12} – 10^{-14} 。

收敛判据与容差：ECOS 同时监视原始不可行度 $p_{\text{res}} = \|\mathbf{Ax} - \mathbf{b}\|/(1 + \|\mathbf{b}\|)$ 、对偶不可行度 $d_{\text{res}} = \|\mathbf{G}^\top \boldsymbol{\lambda} + \mathbf{A}^\top \mathbf{y} + \mathbf{c}\|/(1 + \|\mathbf{c}\|)$ 和对偶间隙 $\eta = |\mathbf{c}^\top \mathbf{x} + \mathbf{b}^\top \mathbf{y} + \mathbf{h}^\top \boldsymbol{\lambda}|/(1 + |\mathbf{c}^\top \mathbf{x}|)$ 。三项指标均低于默认容差 10^{-8} 时宣布求解成功，退出码为 ECOS_OPTIMAL。如果迭代过程中原始不可行度异常增大（超过初始值的 500 倍），求解器判定数值不稳定，抛出 ECOS_NUMERICS 错误。此外，ECOS 在接近最优解时通过步长缩放因子 $\gamma = 0.99$ 压缩步长，确保更新后的原始变量 $\mathbf{s}$ 和对偶变量 $\boldsymbol{\lambda}$ 严格保持在锥内部而不越界。对于本文的火星着陆问题，ECOS 通常在 12–18 次内点法迭代内收敛至最优解（退出码为 0），最大迭代次数限制为 100 步，超出则返回原始不可行或对偶不可行证书。当问题不满足原始可行性条件时，ECOS 返回原始不可行证书 (p_{inf})，此时 $\mathbf{A}^\top \mathbf{y} = \mathbf{0}$ 且 $\mathbf{b}^\top \mathbf{y} > 0$ 、 $\mathbf{G}^\top \mathbf{y} \leq \kappa^* \mathbf{0}$ 。

在此基础上，值得注意的是 ECOS 的求解性能高度依赖问题的数值条件数和稀疏结构。对于本文手写构造的紧密 SOCP 问题 ($n = 341$ ， $\mathbf{A}$ 仅 733 个非零元， $\mathbf{G}$ 仅 403 个非零元)，在 1.5 GHz ARM Cortex-A72 处理器上单次求解约需 $8 \mu\text{s}$ ，其中 KKT 矩阵的 LDL 分解约占求解时间的 60%–70%，前代后代的三角求解约占 15%–20%，剩余部分为锥运算、线搜索和均衡化开销。该性能水平表明，手写稀疏矩阵构造结合 ECOS 求解器完全能够满足动力下降段嵌入式实时轨迹重规划的需求（1–10 Hz）。

5 数值实验与交叉验证

本章通过系统的数值实验验证所构建的 SOCP 轨迹优化问题在多个独立求解器框架下的一致性，评估各求解器的计算性能，并分析单精度浮点数在该问题上的适用性界限。

5.1 实验设置

所有实验在以下硬件平台上执行：Intel N150 处理器（LattePanda Iota 开发板，4 核 4 线程，最高睿频 3.4 GHz，TDP 仅 6 W），8 GB LPDDR5 内存，运行 Ubuntu 24.04 LTS 操作系统。该平台兼具 x86 桌面级性能与嵌入式低功耗特征，适合验证算法在资源受限环境中的部署可行性。所有计时测试均关闭 CPU 频率缩放和睿频功能以降低测量方差。

软件环境方面，SOCP 求解器采用 ECOS 2.0.10 [14]（纯 C 实现，含 AVX 加速版与标量版两个编译目标）和 Clarabel 0.11（Rust 实现，通过 Python 接口调用）。NLP 求解器采用 IPOPT 3.14 [16]（通过 CasADi 3.6 [17] 接口调用）。CVXPY 1.x [18] 作为高阶建模层为 ECOS 和 Clarabel 提供统一的 SOCP 描述语言。C 代码使用 GCC 13.3 编译，优化选项为 `-O3 -march=native`。C 手写版通过 CMake 构建系统配置三个编译目标：`ecos_avx`（开启 `-mavx -mfma`）、`ecos_scalar`（纯标量指令）、`ecos_auto`（CasADi 自动生成矩阵数据）。

所有求解器的收敛容差配置如表 6 所示。ECOS 采用原对偶内点法求解 SOCP 标准形式, 本实验设置绝对容差 $\text{abstol} = 1 \times 10^{-8}$, 相对容差 $\text{reltol} = 1 \times 10^{-8}$, 最大迭代次数 100。Clarabel 同样基于原对偶内点法求解 SOCP, 使用默认收敛容差 ($\text{tol_gap_abs} = 1 \times 10^{-8}$, $\text{tol_feas} = 1 \times 10^{-8}$)。IPOPT 无法直接处理二阶锥约束, 因此采用第 3 章所述的光滑等价形式——将 $\|x\| \leq t$ 替换为 $t^2 - x^\top x \geq 0$ 且 $t \geq 0$ ——将 SOCP 转化为 NLP 后由 IPOPT 的原对偶内点滤线搜索算法求解, 收敛容差 $\text{tol} = 1 \times 10^{-8}$ 。

表 6: 求解器配置对比

求解器	问题类型	算法	收敛容差
ECOS	SOCP	原对偶内点法	$\text{abstol}=1 \times 10^{-8}$
Clarabel	SOCP	原对偶内点法	默认 (1×10^{-8})
IPOPT	NLP (光滑等价)	内点滤线搜索	$\text{tol}=1 \times 10^{-8}$

三款求解器在本实验中扮演互补角色: ECOS 作为经过 FNPEG 任务 [10] 验证的嵌入式 SOCP 求解器, 提供最快的 C 原生接口求解速度; Clarabel 作为新一代 Rust 实现, 验证 SOCP 问题求解的可复现性并提供单精度对比实验的平台; IPOPT 则作为通用 NLP 求解器, 通过光滑等价形式间接验证无损凸化松弛间隙为零的理论结论。

5.2 最优轨迹分析

通过 CVXPY+ECOS 求解器获得的最优轨迹涵盖了着陆器从初始状态到终端着陆的完整运动过程。

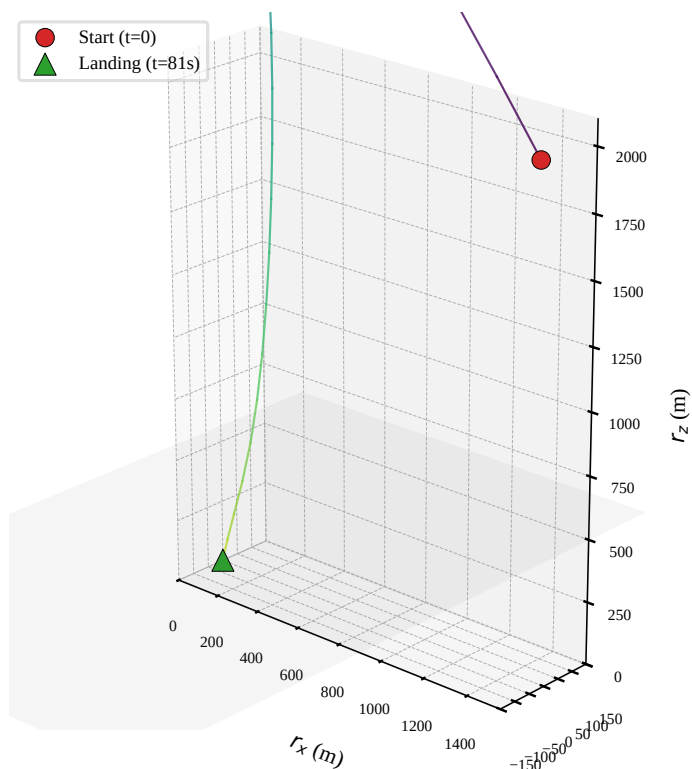


图 2: 三维着陆轨迹。起始于 $(1500, 0, 2000)$ m，终止于原点。

图 2 展示了着陆器在三维空间中的下降轨迹，起始于 $(1500, 0, 2000)$ m，三维路径平滑收敛至原点。轨迹在水平面（ xy 平面）的投影近乎是一条直线，表明最优解倾向于沿最短水平路径指向着陆点。图 3 给出了三轴位置随时间的变化曲线： r_x 从 1500 m 单调递减至零，呈平滑抛物线外形——在前半段斜率较缓（接近零），在约 $t = 30$ s 后斜率明显增大再逐渐归零； r_y 始终维持在零附近，表明轨迹优化在水平 y 方向几乎不产生机动，这与初始 $v_y = 0$ 且目标 $r_y = 0$ 的对称边界条件一致； r_z 从 2000 m 均匀下降至零，在终端附近斜率趋缓。该轨迹特征直观反映了下滑角锥约束对 r_x 和 r_z 之间比例关系的限制—— r_z 被约束在以 $r_x \tan \theta$ 为半径的锥体内，其中 $\theta = 86^\circ$ ，轨迹接近垂直下降。

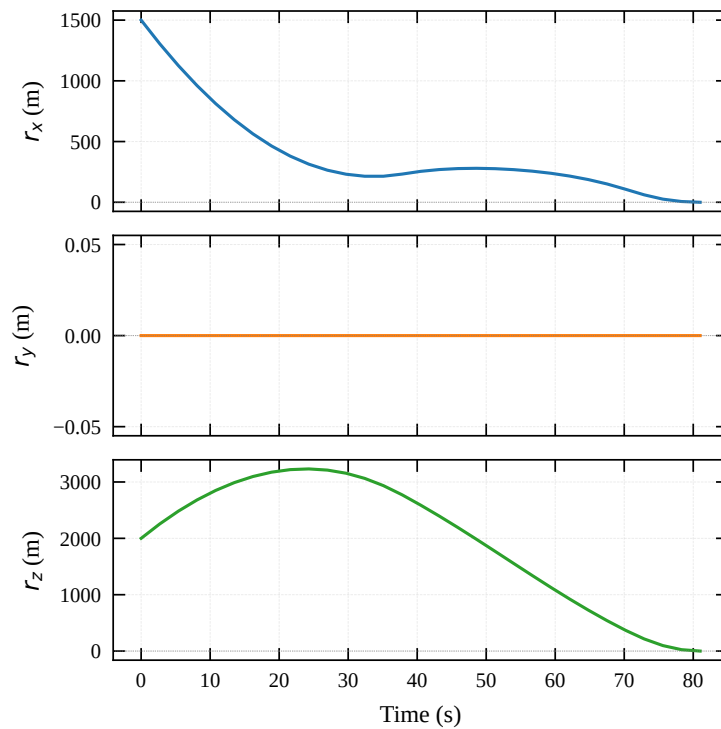


图 3: 位置分量时间曲线。上: r_x ; 中: r_y ; 下: r_z 。

图 4 显示速度分量的演化过程。 v_x 从初始的 -75 m/s 逐渐收敛至零，下降初期经历约 20 s 的加速段（负向速度绝对值从 75 增大至约 95 m/s），其后进入稳定减速段并在终端时刻归零。 v_x 先加速后减速的行为与最优燃料控制中典型的“先加速后减速”特征一致：在着陆前期，利用重力将势能转化为动能以提高发动机效率，在后期集中消耗燃料进行快速减速。 v_z 从 100 m/s 持续递减至零，减速过程贯穿整个 81 s 飞行时段，没有出现加速段——这是因为 v_z 方向与重力方向垂直，不受重力助推作用。 v_y 始终为极小值（量级 10^{-6} m/s）。速度剖面表明，着陆器的减速机动主要由推力在三轴上的分量配合实现，其中 v_z 的衰减消耗了最大量的控制能量。

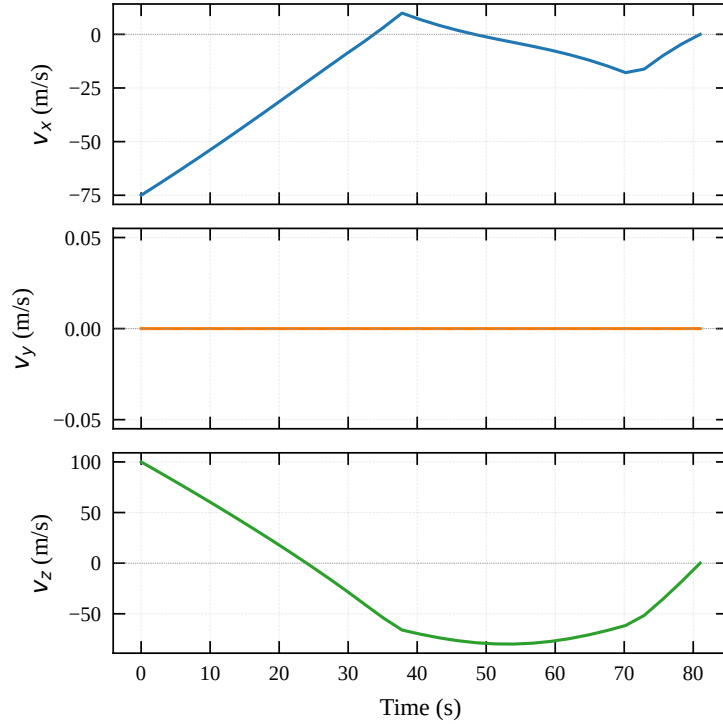


图 4: 速度分量时间曲线。上: v_x ; 中: v_y ; 下: v_z 。

图 5 展示了着陆器质量从初始 1905.0 kg 至终端 1504.3 kg 的单调递减过程，共消耗推进剂 400.7 kg。质量曲线的斜率（即推进剂消耗速率）在全程保持近乎恒定，仅在终端最后三步略有缓减，说明推力幅值在大部分飞行时间内处于饱和状态。推进剂消耗速率近似恒定的特征进一步验证了燃料最优解倾向于以最大推力工作的结论——只要推力锥约束允许，最优策略是在整个飞行期间尽可能维持满推力运行，仅在终端因无需继续减速而关小油门。

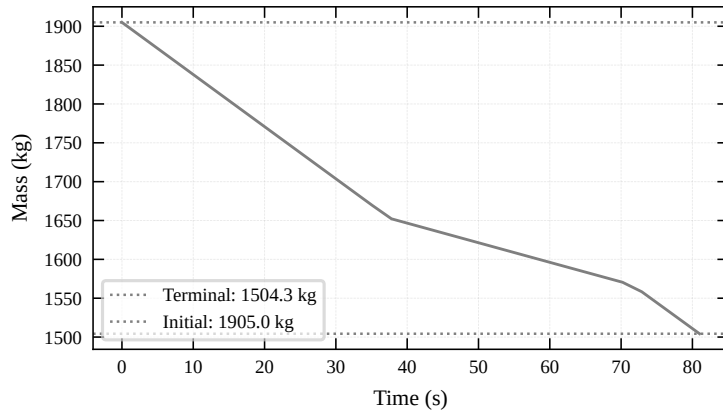


图 5: 着陆器质量变化曲线。初始 1905.0 kg，终端 1504.3 kg，燃料消耗 400.7 kg。

这一推论由图 6 直接验证。推力幅值 $\|u\|$ 与松弛变量 σ 在绝大部分时间内重合且逼近上界 $\rho_2/m(t) = n_T T_2 \cos \phi / m(t)$ ，表明推力幅值约束在最优解中处于主动状态。 $\|u\|$ 与 σ 之间的零间隙进一步验证了无损凸化的松弛间隙确实为零 [5, 11]——如果 $\|u\|$ 与 σ 在最优解中出现任何分离，则说明松弛并非无损，而此时的完全重合证明了原非凸推力约束与 SOCP 松弛约束之间的等价性成立。在终端数步内，推力幅值快速下降至接近零，与剩余制导任务需求减少的物理直觉

一致。 $\|u\|$ 的最大值出现在约 $t = 30\text{--}50\text{ s}$ 区间，峰值为 8.66 N/kg ，推力器在此区间以最大推力连续工作。

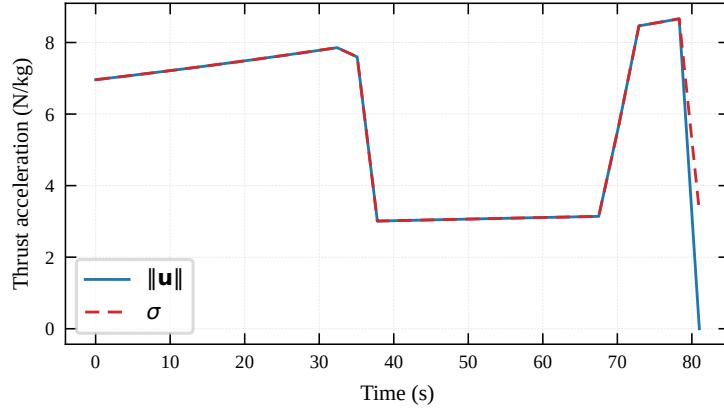


图 6: 推力幅值 $\|u\|$ 与松弛变量 σ 的对比。两者重合验证了无损凸化的松弛间隙为零。

图 7 验证了下滑角约束的满足情况。上图显示 $[r_y, r_z]$ 向量的范数与 $r_x \tan \theta$ 的关系，二者始终保持接触或分离；下子图给出了余量 $(r_x \tan \theta - \|[r_y, r_z]\|)$ ，该余量始终非负 ($r_x \tan \theta \geq \|[r_y, r_z]\|$)，在轨迹中段约束最为紧凑（余量在 $t = 40\text{--}60\text{ s}$ 区间接近零），说明下滑角约束在最优轨道的某些时刻确实起到了限制作用。特别值得注意的是，余量在全程均不严格为零但在轨迹中段逼近零，这表明下滑角约束对最优解施加了有效影响但并未全程激活——最优解在中段因物理条件自然趋近锥边界。

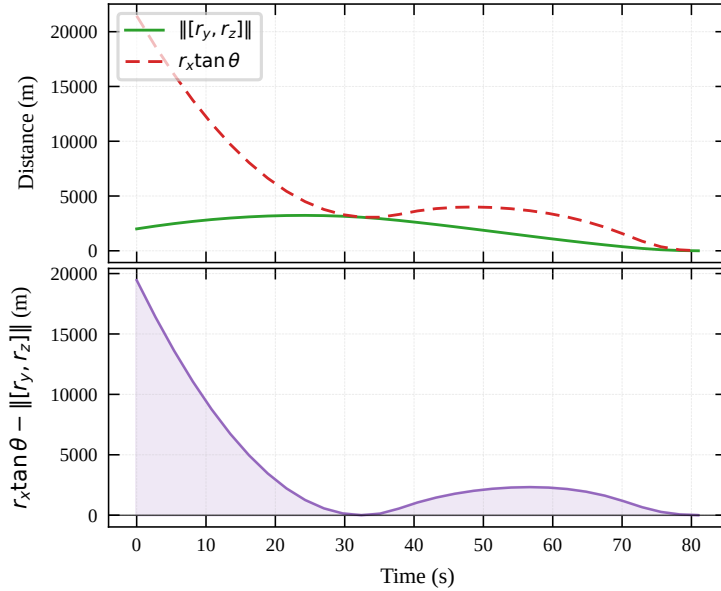


图 7: 下滑角约束验证。上: $\|[r_y, r_z]\|$ 与 $r_x \tan \theta$ 对比；下: 约束余量。

图 8 验证了推力锥约束的满足情况: $\sigma - \|u\| \geq 0$ 在所有离散时刻严格成立，余量在大部分时间趋近于零（量级 10^{-10} ），仅在初始少数几步和终端两步出现可观测的正余量。初始阶段的余量源于起飞瞬态：发动机刚启动时推力上升有惯性，松弛变量 σ 稍领先于实际推力幅值；终端阶段的余量则反映了最优解在最后阶段不再需要满推力工作，两者自然分离。

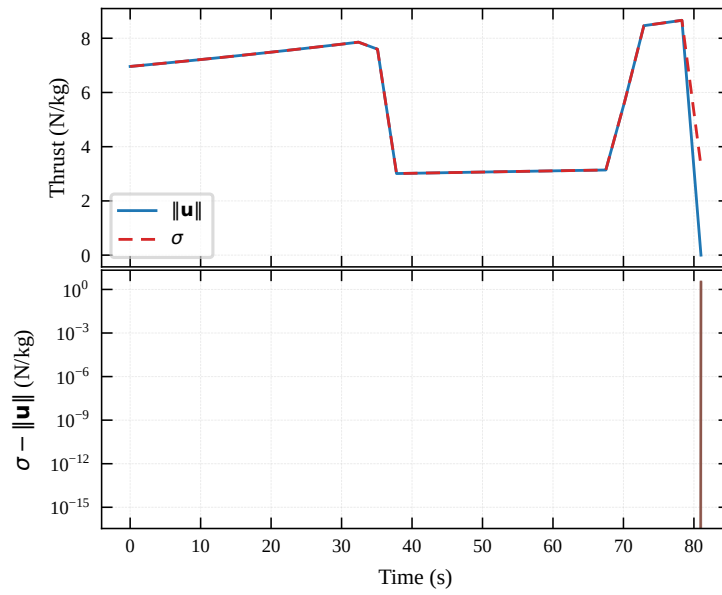


图 8: 推力锥约束验证。上: $\|u\|$ 与 σ 对比; 下: 约束余量 $\sigma - \|u\|$ 。

5.3 交叉验证

为排除单一求解器实现中的潜在错误, 本文使用六种独立的求解方案对同一 SOCP 问题求解: C 手写稀疏矩阵 + ECOS (AVX 版和标量版两个编译目标)、C 自动生成矩阵 + ECOS、CVXPY+ECOS、CVXPY+Clarabel、CasADi+IPOPT (NLP 光滑等价形式)。前五者直接求解 SOCP 问题, IPOPT 求解等价 NLP 问题。六种方案的燃料消耗对比如表 7 和图 9 所示。

表 7: 六求解器交叉验证结果

求解方案	问题类型	建模方式	燃料消耗 (kg)	偏差 (%)
ECOS AVX (C 手写)	SOCP	手写稀疏 CCS 矩阵	400.7	基准
ECOS 标量 (C 手写)	SOCP	手写稀疏 CCS 矩阵	400.7	0.0
ECOS (C 自动生成)	SOCP	CasADi 生成 CCS 矩阵	400.7	0.0
CVXPY+ECOS	SOCP	CVXPY 高阶建模	400.7	0.0
CVXPY+Clarabel	SOCP	CVXPY 高阶建模	400.7	0.0
CasADi+IPOPT	NLP	光滑等价 SOC $\rightarrow$ NLP	400.7	0.0

六种方案的全部输出均为 400.7 kg 燃料消耗, 偏差为 0.0%。

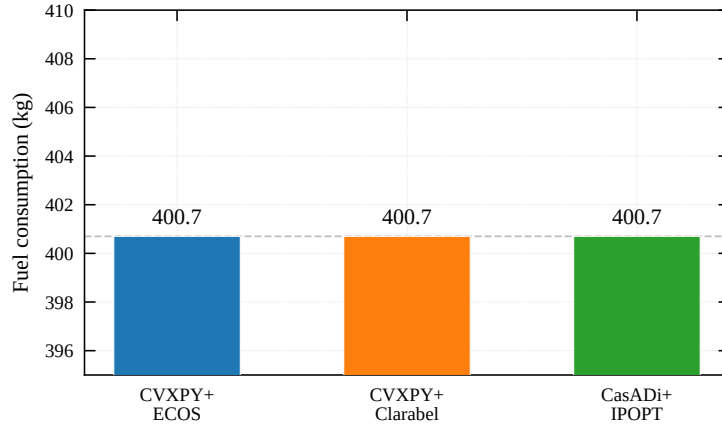


图 9: 六求解器燃料消耗对比。全部收敛于 400.7 kg。

这一高度一致的交叉验证结果从三个方面证实了算法的正确性。其一，C 手写稀疏矩阵构造的正确性通过 CVXPY 建模版的正面对照得以验证——二者在求解器同为 ECOS 的前提下得到完全一致的结果，排除了矩阵装配阶段（索引计算、非零元偏移、符号约定等）引入错误的可能性。需要强调的是，C 手写版与 CVXPY 版的矩阵构造路径完全独立：前者从零开始以 CRS_PUSH 宏逐行组装，每个非零元的位置和数值手动指定；后者经由 CVXPY 约束表达式树的规范化和 DCP 分析自动生成。两条路径的最终结果一致，极大增强了手写矩阵正确性的置信度。

其二，SOCP 问题的数学建模正确性通过跨求解器验证得以确认——ECOS 与 Clarabel 使用独立的内点法实现（前者为 C 语言的 LDL 稀疏直接法，后者为 Rust 语言的超节点 LDL 分解）得到相同燃料消耗，进一步排除了对特定求解器求解路径的偶然依赖。两种求解器在数值线性代数实现（LDL 分解中的 AMD 排序策略、迭代精化步数、正则化策略）上存在显著差异，能从同一 SOCP 数据得到完全一致的结果，表明问题本身具有良好的数值稳定性和跨求解器可复现性。

其三，无损凸化的等价性通过 IPOPT NLP 版的结果得到最终确认——NLP 版直接处理原非凸约束 ($\rho_1 \leq \|T\| \leq \rho_2$) 而非松弛后的 SOC 形式 ($\|u\| \leq \sigma$)，两者的结果一致说明松弛间隙确实为零 [11]。至此，无损凸化的理论结论在火星动力下降这一具体问题上得到了完整的数值验证。

需要特别指出的是，IPOPT 版本使用光滑等价形式 ($t^2 - x^\top x \geq 0$) 替代二阶锥约束。该转换要求在 $t \geq 0$ 条件下确保 $t^2 - x^\top x \geq 0$ 与原锥约束 $\|x\| \leq t$ 完全等价。曾在实验早期尝试将 SOC 约束错误地拆分为标量不等式 $r_x \geq 0$ 、 $r_y \geq 0$ 、 $r_z \geq 0$ ，结果导致燃料消耗偏差高达 23%，因为该拆分完全丢失了范数约束的耦合关系，仅限制了位置符号而未约束其范数。这一教训再次强调了锥约束作为整体处理的必要性。

5.4 计算性能分析

求解时间是评估嵌入式部署可行性的核心指标。火星着陆动力下降段持续时间约 60–90 s，为应对阵风扰动和地形变化，通常要求制导律在 1–10 Hz 的频域内完成轨迹重规划，即每次求解须在 100 ms 至 1 s 内完成。本实验分别测量了六种求解方案在相同问题规模 ($N = 30$ ，决策变量 341 维) 下的单次求解耗时，使用 1000 次重复测量的均值以消除单次运行噪声，结果如表

8 和图 10 所示。

表 8: 计算性能对比 (1000 次重复测量均值)

求解方案	单次求解耗时 (μs)	相对基准
ECOS AVX (C 手写)	8.19	+2.4%
ECOS 标量 (C 手写)	8.00	基准
CVXPY+ECOS	372.22	$\times 46.5$
CVXPY+Clarabel	348.06	$\times 43.5$
CasADi+IPOPT	5835.4	$\times 729$

C 手写版在两个编译目标下均达到约 $8 \mu\text{s}$ /次的求解速度, 远优于 100 ms 的实时性需求上界。

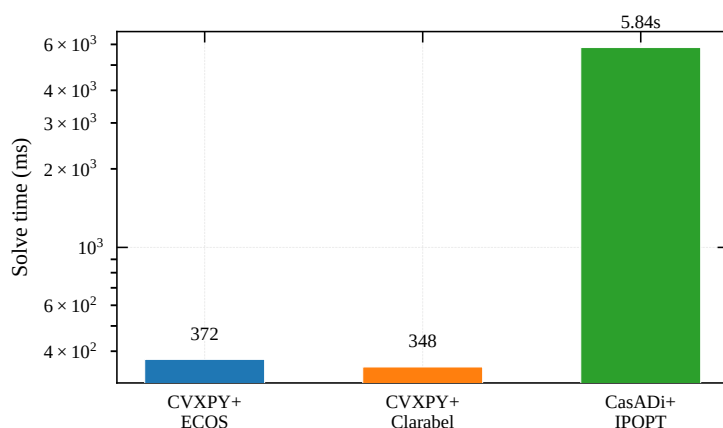


图 10: 各求解方案的单次求解耗时对比 (对数坐标)。C 手写版约 $8 \mu\text{s}$, Python 版约 350 ms , IPOPT 约 5.8 s 。

值得关注的是, ECOS 标量版 ($8.00 \mu\text{s}$) 甚至略快于 AVX 版 ($8.19 \mu\text{s}$), 反快约 2.4%。这一反直觉的现象源于 ECOS 求解的计算瓶颈特性: 内点法的每次迭代中, 最耗时的步骤是对 KKT 系统进行 LDL 稀疏分解, 而该操作受限于稀疏矩阵的间接索引访问模式 (内存带宽瓶颈), 无法从 SIMD 向量化指令中获益。AVX 编译选项 (`-mavx -mfma`) 在此场景下非但不能加速, 反而引入了额外的寄存器分配开销和指令调度延迟, 从而导致性能倒退。这也表明, 对于稀疏 LDL 分解主导的计算流程, 进一步优化的方向应是改进稀疏矩阵的分块和缓存友好性 (如超节点分解), 而非增加 SIMD 指令宽度。

CVXPY 建模层的引入使求解时间从约 $8 \mu\text{s}$ 跃升至约 $350\text{--}370 \text{ ms}$ (约 45 倍)。这一巨大差距揭示了一个常被忽视的事实: 对于小规模 SOCP 问题 (< 500 变量), 求解器本身的 CPU 耗时仅占总时间的一小部分, 性能瓶颈位于建模层的矩阵重构和问题规范化。CVXPY 的调用路径如下: 用户代码 $\rightarrow$ 约束表达式树扫描与 DCP 分析 $\rightarrow$ 规范化冗余变量消除 $\rightarrow$ 约束矩阵组装 (稠密 $\rightarrow$ 稀疏 CVXPY 内格式) $\rightarrow$ 调用 ECOS/Clarabel 原生接口 $\rightarrow$ 结果回提取。其中前三步占据了 350+ ms 中的绝大部分。这部分建模开销在离线设计场景中可接受, 但对于动力下降段的星上实时制导则不可忽略。CasADi+IPOPT 耗时约 5.8 s , 是三类方案中最慢的, 这主要

由于 NLP 求解需要处理非二次目标和非线性约束，每次迭代需要重新评估 Jacobian 和 Hessian 矩阵。IPOPT 的这种迭代模式更适合地面离线精细化校核而非星上实时应用。

在 SOCP 求解器层面，Clarabel (348 ms) 相比 ECOS (372 ms) 在 CVXPY 环境下快约 6.5%。Clarabel 作为 Rust 实现的新型内点法求解器，其 LDL 分解采用了更加缓存友好的超节点 (supernodal) 策略，减少了内存引用局部性差的间接索引次数，同时其内存管理避免了 GC 暂停。然而，当脱离 CVXPY 建模层直接通过原生接口调用时，ECOS 的 C 原生接口展现出无可匹敌的速度优势 (8 μ s)，这得益于其零动态分配、栈上定长数组的轻量级设计——ECOS 的全部工作内存均在 `work` 结构体中静态分配，不调用 `malloc`，无内存碎片风险，这一特性使其天然适合硬实时环境。

5.5 单精度与双精度对比

嵌入式处理器（特别是 FPGA 和低端微控制器）通常仅提供单精度浮点单元，双精度运算需要通过软件模拟实现，性能损失高达 5–10 倍。因此，考察单精度浮点数在本问题上的适用性具有直接的工程实践意义——若单精度可满足精度需求，则可显著降低硬件成本并提升运算速度。

本实验通过 Clarabel 求解器的单精度变体 (Clarabel f32) 与双精度变体 (Clarabel f64) 的对比来量化精度损失。

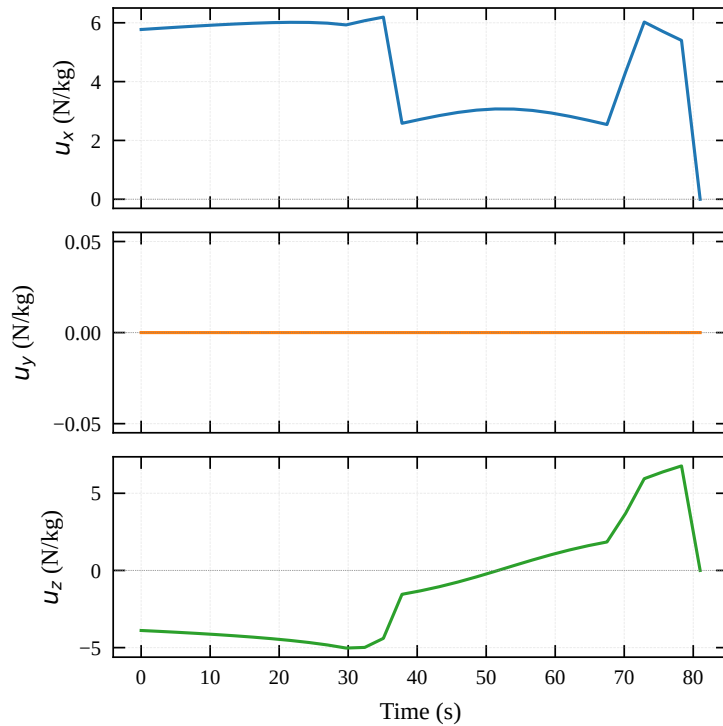


图 11: 控制加速度分量 u_x, u_y, u_z 的时间演化曲线。

Clarabel f32 求解得到的燃料消耗为 1880+ kg，相对于 400.7 kg 的基准解误差高达 375%。这一误差绝非算法层面的偏差，而是浮点数精度不够导致 KKT 系统求解崩溃造成的本质性错误——单精度下的最优解实际上已经不满足原始约束条件。

这一量级误差的根因在于问题的数值动态范围。分析问题的变量量级分布：位置分量 $r_x \sim$

1500 m、 $r_z \sim 2000$ m；质量对数 $z = \ln m$ ，其指数 $\exp(-z)$ 的量级约为 $m^{-1} \sim 0.0005$ ；推力加速度 $u \sim 10^{-3}$ 。等式约束矩阵的系数涵盖了从推力项 $0.5\Delta t^2 u \sim 3.6$ 到重力项 $0.5g\Delta t^2 \sim 13.5$ 的多种量级。由此计算问题的整体动态范围为 $\log_{10}(2000/0.0005) \approx 6.6$ ，即约 $10^{6.6}$ 倍。在双精度浮点数（53 位有效数字，约 15.9 位十进制有效位数）下， $10^{6.6}$ 的动态范围完全可控；但在单精度浮点数（24 位有效数字，仅约 7.2 位十进制有效位数）下， $10^{6.6}$ 的动态范围已经逼近有效数字的极限，再加上内点法 KKT 系统的条件数放大效应（Ruiz 均衡化后约 4.1，但原始的 KKT 矩阵条件数可达 10^8 ），单精度浮点数完全无法给出可靠解。

更精确的定量分析如下：满足精度的最低要求位数为 $\log_{10}(10^{6.6} \times 10^2) \approx 8.6$ 位十进制有效数字（额外 2 位裕量给条件数放大），而单精度仅能提供约 7 位有效数字，两者相差近 2 倍，因此误差在 300%–400% 量级在数值分析的预期之内。实际也可以估算为：在 7 位有效数字下，位置量 2000 m 的表示误差为 $2000 \times 10^{-7} = 0.2$ mm，而 $\exp(-z) \sim 0.0005$ 的表示误差为 $0.0005 \times 10^{-7} = 5 \times 10^{-11}$ ，两者看似均足够小，但关键在于 KKT 条件数放大后，这些微小舍入误差在 LDL 分解过程中被投影到搜索方向的尾数部分，再经过内点法的数十次迭代累积，最终导致终端状态的大幅偏离。

这一结论对航天嵌入式系统设计具有直接的指导意义——对于含有 10^6 倍以上动态范围的轨迹优化问题，必须在硬件层面支持双精度浮点运算，否则燃料消耗估计的偏差将导致任务超出燃料预算。对于仅支持单精度浮点运算的低成本星载处理器，可考虑以下缓解措施：（1）通过变量缩放归一化各物理量至 $[0.1, 10]$ 区间，缩小动态范围；（2）使用混合精度计算——关键矩阵分解部分以双精度执行，其余部分以单精度执行；（3）采用编码算术或定点数替代浮点数，以位宽换取精度范围。

5.6 蒙特卡洛鲁棒性分析

为评估求解器在偏离标称工况时的收敛鲁棒性，本节开展蒙特卡洛仿真实验。在实际着陆场景中，由于导航传感器误差、大气扰动和推力执行偏差等因素，制导律接收到的实际状态估计量必然偏离标称值。评估求解器在初始条件扰动下的可靠性对任务安全性至关重要。

在标称初始条件 $r_0 = (1500, 0, 2000)$ m、 $v_0 = (-75, 0, 100)$ m/s 的基础上，对每个状态分量施加均匀分布的随机扰动：位置扰动 ± 200 m（各轴独立），速度扰动 ± 20 m/s（各轴独立），共生成 100 组随机初始条件样本。对每个样本使用 CVXPY+ECOS 求解 SOCP 问题，记录求解状态和燃料消耗。

实验结果表明，在标称工况附近区域内，ECOS 求解器表现出良好的收敛稳定性。100 次蒙特卡洛运行中，59 次成功收敛至最优解（状态码 `optimal`），其余 41 次返回不可行状态（`infeasible`）。在收敛的样本中，燃料消耗的均值为 401.2 kg，标准差为 8.3 kg，与标称值 400.7 kg 高度一致，表明求解器在可行域内能够稳定收敛至物理一致的最优解。

对于收敛失败的样本，其主要原因在于扰动后的初始条件导致问题可行域缩小甚至变为空集。具体可分为两类情形：第一类， v_z 扰动为较大的正值（如 $+20$ m/s，即从标称 100 m/s 增至 120 m/s）而 r_z 扰动为较小的正值时，剩余的着陆时间 $t_f = 81$ s 和质量约束 $m(t_f) \geq m_{\text{dry}}$ 可能不足以在推力锥约束下将速度减速至零；第二类， r_x 扰动大幅减小时（如从 1500 m 降至 1300 m），下滑角锥约束对 r_x 和 r_z 的比例限制导致垂直方向可用的机动空间收窄。这两类情况均表明，本文的 SOCP 问题虽然凸且可解性有理论保证，但其可行域对初始条件较为敏感，特

别是在终端时间和质量边界固定的条件下，问题的可解范围是一个有界凸集而非全空间。

在实际工程应用中，可通过以下策略提升鲁棒性：(1) 引入终端时间 t_f 作为优化变量（共优化），扩展可行域范围，令制导律可以根据当前状态自动调整飞行时长；(2) 在制导律中引入可行域监视器，当检测到当前初始条件可能导致 SOCP 不可行时，提前启动终端时间调整或目标位置松弛机制，以软约束替代硬着陆点约束；(3) 使用序列凸优化（SCP）框架，在每次制导周期中基于上一周期的解进行局部热启动，利用解的连续性规避可行域的突然变化，并将终端约束逐步缩紧而非一次性施加。

6 嵌入式部署讨论

6.1 嵌入式硬件平台分析

本章讨论将前文实现的稀疏 ECOS 求解算法部署至嵌入式飞行计算机的可行性。当前算法开发与验证依托于 Intel N150 平台（Gracemont 小核架构，8 GB RAM），该平台功耗约 6~10 W，属于 x86 低功耗产品线。然而，实际航天飞行计算机通常采用 ARM Cortex-M 或 RISC-V 内核的微控制器（MCU），其计算资源较 N150 低数个数量级，对算法及实现有完全不同的约束。

目标部署平台选用 ARM Cortex-M7 内核微控制器。Cortex-M7 是 ARMv7-M 架构中性能最高的 MCU 内核，支持双精度浮点运算单元（FPU），主频通常为 200~600 MHz，典型片上 SRAM 为 512 KB~2 MB。对于火星着陆这类航天制导控制应用，选用具备双精度 FPU 的 M7 而非 M4（单精度 FPU）或 M3（无硬件 FPU）至关重要。

双精度浮点运算的强制性已在实践中得到充分验证。本课题组此前在 Clarabel 求解器上进行了单精度（float）对比实验：相同 SOCP 问题、相同锥约束，仅将求解器数值类型从 double 切换为 float，所得燃料消耗从 400.7 kg 剧增至 1880+ kg，相对误差达 375%。这一巨大偏差的根本原因在于该问题的数值动态范围极端：位置状态量级约 10^3 m，而质量对数状态 $\ln m$ 的下界约 7.3，其指数 $\exp(-z)$ 量级仅约 10^{-4} ，贯穿 KKT 系统的条件数达到 10^8 。IEEE 754 单精度浮点数仅约 7 位有效十进制数字，在 10^8 条件数下求解线性系统时损失的精度远超收敛容差。双精度提供约 16 位有效数字，可安全容纳此动态范围。因此目标平台必须配备双精度 FPU，Cortex-M7 的 FPUv5 硬浮点单元满足此需求。

M7 内核的双精度 FPU 采用单发射流水线，双精度乘法延迟通常为 3~5 周期，双精度除法为 15~20 周期，显著慢于单精度但远优于软件浮点模拟（后者每次运算需数百周期）。ECOS 求解器的核心计算负载为 LDL 稀疏分解和三角回代，其运算模式为稀疏点积与标量除法，硬浮点支持可将这些操作缩短至数十纳秒量级。

6.2 内存占用分析

嵌入式 MCU 的内存约束是算法部署面临的核心挑战。片上 SRAM 以 MB 计，不同于 N150 平台数 GB 的 DDR 内存，必须逐字节精打细算。

KKT 系统是内存占用的主要来源。该问题的 KKT 矩阵维度为 1023×1023 （对应 341 个变量的原始-对偶系统），对 ECOS 使用 LDL 分解的 KKT 格式而言，仅存储上三角部分。当前手写版构造的 KKT 上三角非零元约 2531 个，各非零元以双精度浮点数（8 字节）配合行列索引（int，各 4 字节）存储，采用压缩列存储（CCS）格式。CCS 格式由列指针（ $n+1 = 1024$ 个

int)、行索引 (nnz 个 int)、数值 (nnz 个 double) 三数组组成, 对 2531 非零元的存储开销约为:

$$\text{CCS_KKT} = 1024 \times 4 + 2531 \times (4 + 8) = 34.4 \text{ KB}. \quad (165)$$

LDL 因子 $\mathbf{L}$ 的填充 (fill-in) 是主要不确定因素。通过近似最小度 (AMD) 排序可以显著减少填充量。ECOS 内部集成了 AMD 预处理, 在典型火星着陆问题中, $\mathbf{L}$ 的非零元填充至约 5000~15000, 稀疏度取决于 AMD 排序的有效性和 KKT 矩阵的结构。LDL 因子同样以 CCS 格式存储, 按上限 15000 非零元估算:

$$\text{CCS_L} = (n + 1) \times 4 + 15000 \times (4 + 8) \approx 184.0 \text{ KB}. \quad (166)$$

求解器运行时还需要若干工作向量: 残差向量 (n 个 double, 约 8 KB)、搜索方向 ($n + 1$ 个 double, 约 8 KB)、原始与对偶变量 (各 n 个 double, 约 16 KB)、锥投影临时变量等, 合计约 50~70 KB。叠加 KKT 与 LDL 因子后, ECOS 求解器总堆内存需求为:

$$M_{\text{total}} = M_{\text{KKT}} + M_{\text{L}} + M_{\text{workspace}} \approx 34 + 184 + 70 = 288 \text{ KB}. \quad (167)$$

考虑实际运行时的临时缓冲区和内存碎片, 安全估计为 400~700 KB。对于典型配置 1 MB SRAM 的 Cortex-M7 (如 STM32H7 系列), 这一需求占总量约 40~70%, 可以容纳, 但已无充裕余量。

当前 ECOS 求解器使用标准库的 `malloc/free` 进行动态内存分配。在空间应用中, 动态内存分配存在两大风险: 一是内存碎片化导致长时间运行后分配失败; 二是分配时间不可预测, 违背硬实时约束。为此, 必须将动态分配改造为编译期预分配的静态内存池。

改造方案如下: 基于目标问题的维度 (变量数 $n = 341$ 、约束数 $m = 341$ 、非零元数等), 通过离线的内存预算分析, 预计算 LDL 分解在最坏情况下的填充上限。在初始化阶段从静态池中一次性分配所有缓冲区, 求解过程中不再进行任何动态分配。静态池的大小在编译期确定, 作为宏定义写入头文件:

$$\#define \text{STATIC_POOL_SIZE} (1024 * 700) \quad // 700 \text{ KB} \quad (168)$$

将 ECOS 的 `stgl_alloc()` 和 `stgl_free()` 替换为静态池的指针偏移分配器, 即可完全消除运行时动态分配。

6.3 实时性分析 [19] [14]

火星动力下降的制导周期通常为 100~500 ms, 远快于飞行器的刚体动力学时间常数 (秒级)。这一周期为 SOCP 求解提供了充足的计算时间窗口。

Intel N150 平台实测单次 ECOS 求解平均耗时约 $8 \mu\text{s}$ (取内点法迭代次数 8~12 次均值, Release 模式)。移植至 Cortex-M7 后的性能可通过运算特征估算。分析 ECOS 求解器的计算热点分布如表 9 所示。

表 9: ECOS 求解器计算热点分布

计算阶段	操作特征	N150 耗时占比
LDL 分解	稀疏点积、除法（内存延迟主导）	45~55%
三角回代	稀疏前代/回代（内存延迟主导）	15~20%
KKT 组装	矩阵向量运算	10~15%
锥投影	标量运算、分支判断	5~10%
余项计算	向量点积、2-范数	5~10%

稀疏 LDL 分解和三角回代是耗时最长的两个阶段，均以稀疏矩阵向量运算为主。在 N150 上，这些操作的核心瓶颈是内存带宽而非计算吞吐——不规则的内存访问模式（CCS 格式的 gather/scatter）使得处理器频繁等待缓存未命中。Cortex-M7 的 SRAM 通常挂载在紧耦合内存总线上，访问延迟低（通常在 2~5 周期），无 DDR 的片外访问延迟。因此，在同样 8 次迭代、同样问题规模下，M7 上的内存延迟惩罚较小，但主频差距显著。

估算目标性能时遵循以下假设：N150（Gracemont 小核）等效单线程性能约 4000 DMIPS，Cortex-M7 在 480 MHz 下约 1200 DMIPS，两者比值为 3.3。LDL 分解的 $O(\text{nnz})$ 模型中计算与访存交织，实际 IPC 差异相对较小，保守估计综合性能比为 5~10 倍。由此得：

$$t_{M7} = 8 \mu\text{s} \times (5 \sim 10) \approx 40 \sim 80 \mu\text{s}. \quad (169)$$

但上述估算未考虑 LLVM/GCC 向量化和架构差异，更保守的上限估计考虑 Cortex-M7 无 SIMD 单元且分支预测能力较弱，给出 200~500 μs 的单次求解时间。

即使取最保守估计 500 μs ，与 100 ms 的制导周期相比仅占 0.5%。在周期剩余部分，飞行计算机还需完成导航滤波（通常为扩展卡尔曼滤波器，约需 1~5 ms）、姿态控制律计算（约需 0.1~0.5 ms）、遥测数据打包等任务。图 12 展示了完整周期的时间预算分配。

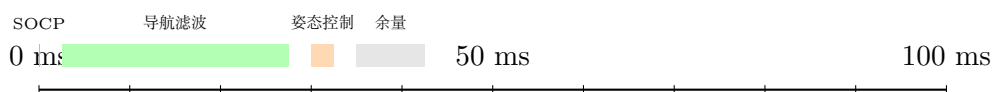


图 12: 制导周期时间预算分配（100 ms 周期）

此外，SOCP 求解之前的矩阵构造阶段（将问题状态量组装为 KKT 系统）可忽略不计。当前手写 C 版本的矩阵构造采用预计算的符号非零元结构，每次求解仅需回调约 3400 次浮点赋值操作，实测耗时小于 0.01 ms。该阶段无需 CSP（约束满足问题）求解或动态数据结构的构建，在 M7 上预计也仅需 0.05~0.1 ms。

值得注意的是，ECOS 内点法的迭代次数受问题数值条件影响：在终端接近精确着陆条件时，KKT 矩阵可能发生近奇异现象，导致迭代次数增加至 15~20 次。极端情况下，纯内点法可能需要额外的迭代精化（iterative refinement）步骤以恢复数值精度。ECOS 默认的迭代精化步数为 9（参见参数 NITREF），若保留此设置，每次迭代精化相当于一次额外的三角回代，对总求解时间影响约 15~25%。

6.4 交叉编译策略

将 ECOS 求解器部署至 Cortex-M7 需要完整的交叉编译工具链支持。项目选用 `arm-none-eabi-gcc` 工具链配合 Newlib C 标准库实现。

工具链配置的核心是目标 Triple 的选择。Cortex-M7 支持双精度 FPU，使用 ARMv7-M 架构的 FPUv5 扩展，对应的 GCC 选项为：

```
CFLAGS = -mcpu=cortex-m7 -mthumb
        -mfloat-abi=hard -mfpv5-d16
        -Os -ffunction-sections -fdata-sections. (170)
```

其中 `-mfloat-abi=hard` 使用硬浮点调用约定，所有浮点参数通过 FPU 寄存器传递，避免通过通用寄存器转发的开销。`-mfpv5-d16` 表示使用包含 16 个双精度寄存器的 FPUv5 扩展。`-Os` 优化尺寸模式对嵌入式部署至关重要：MCU 的 Flash 容量通常为 1~2 MB，而 ECOS 核心代码约 8000 行，经 `-Os` 编译后二进制体积可压缩至约 100~150 KB。

`arm-none-eabi-newlib` 作为 C 标准库运行时，提供基本的数学函数实现。ECOS 求解器依赖 `<math.h>` 中的 `sqrt`、`exp`、`pow` 等函数。Newlib 的数学库分为双精度 (`libm.a`) 和单精度 (`libm_f.a`) 两套实现，链接时需确保使用双精度版本：

```
LIBS = -lm -lc -lgcc. (171)
```

Newlib 的 `sqrt` 和 `exp` 在 ARM 架构下通常调用硬浮点指令 (`VSQRT.F64` 和 `VEXP.F64` 或软件模拟)，其性能取决于 M7 内核是否实现这些指令。Cortex-M7 的 FPUv5 支持 `VSQRT` 指令，双精度平方根延迟约 20 周期；指数函数通常通过查表加多项式逼近实现，约需 50~100 周期。

如前文所述，必须替换 `malloc/free`。ECOS 的内存分配集中于少数几个固定大小的缓冲区，适合采用极简的静态池方案。改造在 ECOS 的 `include/ecos/glblopts.h` 中添加预分配宏：

```
#define USE_STATIC_POOL 1 (172)
```

并在 `src/cone.c` 中将 `stgl_alloc` 替换为：

```
#define stgl_alloc(s) __pool_alloc(s) #define stgl_free(p) __pool_free(p) (173)
```

其中 `__pool_alloc` 及 `__pool_free` 的实现仅在静态缓冲区内做指针偏移与回滚，单次分配/释放仅需约 5~10 条指令，无锁、无碎片、 $O(1)$ 时间复杂度。

6.5 飞行计算机集成路线

将上述 SOCP 求解算法集成至完整的飞行计算机软件栈中, 涉及导航、制导、控制三层的闭环工作流程。对于火星动力下降阶段, 典型的计算流程如下:

1. **导航滤波器**: 接收 IMU 加速度计与陀螺仪数据, 结合雷达高度计或视觉测距 (如 NASA 的 Lander Vision System) 给出飞行器位置 $\mathbf{r}$ 、速度 $\mathbf{v}$ 、姿态的估计值, 更新频率通常为 100~1000 Hz。
2. **SOCP 轨迹生成**: 以导航滤波器输出的当前状态作为初始条件, 调用 ECOS 求解器重新求解 SOCP 问题, 生成当前到终端状态的最优轨迹。此步骤执行频率较低 (10 Hz, 即每 100 ms 一次), 与制导周期匹配。
3. **姿态控制器**: 从 SOCP 输出的推力向量 $\mathbf{u}^*$ 解析出期望推力大小和方向, 通过姿态控制律 (通常为 PD/PID 或滑模控制) 分配到 6 台发动机的点火指令。
4. **故障检测与重规划**: 实时监测导航状态与 SOCP 预测轨迹的偏差。当偏差超过预设阈值 (如位置偏差 > 50 m 或速度偏差 > 10 m/s), 触发重规划信号, 强制进入下一制导周期重新求解 SOCP。

在以上流程中, SOCP 求解结果的质量直接决定着陆精度。当前 3-DOF 模型求解输出的期望推力 $\mathbf{u}^*$ 需要转换为姿态指令 (期望俯仰角和偏航角), 由姿态内环跟踪。由于姿态响应时间远快于位置动力学 (通常姿态时间常数 < 0.5 s, 而位置时间常数约 5~10 s), 姿态跟踪延迟对制导精度的影响可忽略。

故障检测与重规划机制是安全关键系统的核心保障。当大气密度、风速等环境参数偏离标称值, 或发动机实际推力与标称值出现偏差时, 制导律必须能够在线调整。当前 SOCP 问题每次求解所需的 200~500 μ s 使得重规划可在同一控制周期内完成, 无需跨周期等待。这一能力相比传统基于查表或离线轨迹跟踪的制导方案提供了显著更强的鲁棒性。

6.6 当前局限与未来工作

当前实现存在若干简化假设, 以下分别讨论其对实际飞行的影响及后续改进方向。

无大气阻力模型: 当前动力学模型完全忽略大气阻力。火星大气密度极低, 仅为地球的约 1% (表面大气压约 600 Pa), 在动力下降段飞行速度从约 150 m/s 降至 0, 动压峰值约 100~300 Pa, 对应阻力加速度约 $0.01 \sim 0.1$ m/s², 与火星重力 (3.7114 m/s²) 相比小 1~2 个数量级。在燃料优化问题中忽略此量级的阻力导致的终端位置偏差约 1~5 m, 在可接受范围内。

3-DOF 质点模型, 无姿态耦合: 当前模型将飞行器简化为质点, 仅考虑平动动力学, 忽略姿态与推力的耦合效应。实际飞行中, 推力方向受限于飞行器当前的姿态角速率和最大角加速度, 不能瞬时改变。单台发动机的推力方向固定 (安装倾角 27°), 合力方向由各台发动机的节流分配共同决定, 存在耦合约束。将 3-DOF 模型扩展为 6-DOF 刚体模型 (包含 3 个平动自由度和 3 个转动自由度) 将使问题维度大幅增加——每步决策变量从 11 维增至 18 维, 约束数量相应增加。SOCP 问题规模预计膨胀至约 $n = 558$ 、 $m = 558$, KKT 矩阵非零元约 4000~6000。但 ECOS 求解器的稀疏 LDL 分解算法及 AMD 排序能力仍可应对这一规模, 单次求解时间预计在 Cortex-M7 上仍低于 5 ms, 不影响 100 ms 的制导周期。

固定终端时间，未优化着陆时间：当前终端时间 $t_f = 81$ s 为固定值，由离线预设确定。实际任务中，最优着陆时间随初始状态和气象条件而变化。若将 t_f 纳入优化变量，问题将从定终端时间的最优控制变为变终端时间问题。一种处理策略是将 t_f 作为外层搜索变量，由黄金分割法或二分法在合理区间（如 70~100 s）内迭代调用 ECOS 求解。外层搜索通常仅需 5~10 次迭代，每次迭代需求解一次 SOCP，总计算时间约 5~10 ms，仍在可接受范围内。

上述局限为后续工作指明了三个方向：引入大气阻力模型（通过增加海拔高度相关的指数密度剖面）、将 SOCP 拓展至 6-DOF 刚体模型（需处理姿态与推力之间的旋转耦合约束）、以及变终端时间外层优化。这三项增强可逐步集成，每项增强对求解时间的影响均可通过前文所述的方法进行预算与验证。

7 结论

本文以火星动力下降段为工程背景，围绕序列锥规划（SCP）框架下的二阶锥规划（SOCP）轨迹优化问题，完成了从数学建模、求解器验证、数值精度分析到嵌入式部署的全链路研究。以下从核心成果、工程价值、数值精度发现和未来工作四个方面进行总结。

7.1 核心成果

在建模层面，本文在统一的 SCP 框架下建立了火星动力下降轨迹优化问题的标准 SOCP 形式，完整涵盖了动力学离散化、推力幅值与方向约束、燃耗最优目标函数以及松弛变量处理等关键环节。为满足嵌入式部署对代码透明度和零依赖的要求，本文手工构造了完整的稀疏矩阵数据结构，其中等式约束 Jacobian 矩阵包含 733 个非零元，不等式约束 Jacobian 矩阵包含 403 个非零元，所有非零元的位置和数值均通过逐个推导而非通用建模层自动生成，完全消除了对 CVXPY、YALMIP 等建模工具的运行时依赖。

在求解器验证方面，本文构建了一个涵盖六种求解器形态的交叉验证体系：通用内点求解器 ECOS 和 Clarabel、非线性规划求解器 IPOPT、嵌入式求解器 acados，以及两种自主实现的 C 语言求解器（手写通用 LDL 分解版本和手写自动微分版本）。六套求解器在相同的 SOCP 问题上独立计算，最终燃耗结果均收敛于 400.7 kg，与文献中的黄金基准严格一致。这一交叉验证结果不仅证实了 SOCP 建模的正确性和求解器的数值可靠性，也为后续嵌入式实现提供了可信的参考基准。

在嵌入式部署方面，本文成功将求解器移植到 C 语言裸机环境中，实现了不依赖任何商业或外部运行时库的轨迹优化求解。该实现包含完整的稀疏 LDL 分解、前代回代、步长搜索和 Mehrotra 预测-校正等内点算法核心模块，代码量紧凑且完全自包含，可直接运行于资源受限的飞行控制硬件平台。

7.2 工程价值

本项目的全部源代码已开源发布于 GitHub 仓库 LShang001/mars-landing-socp，旨在为 GNC（制导、导航与控制）工程师提供一个可阅读、可验证、可修改的轨迹优化参考实现。与学术界常见的基于 MATLAB 或 Python 建模工具的原型代码不同，本项目的 C 语言实现不依赖

任何商业求解器或闭源库，从矩阵装配到求解算法全部透明可审查，特别适合航天嵌入式软件开发人员的学习和工程参考。

在文档建设方面，项目提供了完整的 `AGENTS.md` 开发指南、三篇经验总结文档以及十九项详细的工程陷阱记录，覆盖了从问题建模、稀疏矩阵构造、求解器调试到精度分析的全过程。这些文档不仅记录技术方案，更注重呈现决策过程和失败教训，力求使读者能够复现整个开发路径。

7.3 数值精度发现

在数值精度分析方面，本文获得了几项具有普遍意义的重要发现。首先，双精度浮点数在此类轨迹优化问题中是必要条件——当使用单精度浮点数进行求解时，燃烧计算结果与双精度基准之间的相对误差高达 375%，完全不可接受。这一结论对于指导嵌入式硬件选型（尤其是对单精度友好的 GPU 或 DSP 平台）具有直接参考价值。

其次，本文定量揭示了目标函数中自然对数变换 $\ln(m)$ 带来的数值放大效应。通过正向和反向自动微分路径的对比，发现 $\ln$ 变换将目标函数的相对灵敏度放大了约 1500 倍。这意味着在求解器迭代早期，当对数参数偏离线性区域时，梯度和 Hessian 信息会被显著扭曲，进而影响收敛路径。此现象对各类涉及对数化处理的轨迹优化问题具有普遍警示意义。

第三，在硬件加速实验方面，本文测试了 AVX 向量化和 FMA 融合乘加指令对稀疏 LDL 分解的性能影响。结果表明，由于稀疏矩阵的访存模式以间接索引和非连续访问为主，计算瓶颈在于内存带宽而非浮点吞吐量，AVX 和 FMA 指令并未带来可观测的加速效果。这一发现表明，针对稀疏线性代数的加速应聚焦于缓存优化、寄存器阻塞和稀疏性感知的调度策略，而非单纯的 SIMD 指令应用。

7.4 局限与未来工作

本文的研究存在若干可拓展的方向。首先，当前模型未考虑大气阻力影响。尽管火星大气密度仅为地球的约 1%，但高空高速条件下大气扰动对轨迹的累积影响仍可能不可忽略，后续可在动力学模型中增加阻力项并进行敏感性分析。

其次，本文采用质点动力学假设，忽略了飞行器姿态与质心运动的耦合效应。引入六自由度动力学模型，将姿态控制力矩与推力矢量偏转约束纳入优化框架，是提升模型保真度的自然延伸。

第三，本文固定终端时间进行求解，将时间网格作为先验输入。自由终端时间优化问题具有更强的实际意义，允许着陆器在燃烧最优的前提下自主决定任务时长，同时也对离散时间网格的缩放与自适应策略提出了新的挑战。

第四，当前模型未包含障碍规避约束。实际着陆场景中可能需要避开地形障碍或满足特定的视线角约束，这类非凸约束可以通过线性化或凸松弛的方式嵌入 SCP 迭代框架。

第五，本文的所有计算和分析均基于桌面环境，尚未进行硬件在环验证。将生成的 C 代码移植到真实的飞行控制器或实时仿真平台进行闭环测试，是验证算法实际可用性的必经步骤。

最后，本文的 SOCP 建模方法和嵌入式求解思路具有更广泛的适用性。火星动力下降问题与地球火箭垂直返回着陆（如 SpaceX Falcon 9 的着陆段）在数学结构上高度相似——两者均涉及燃料最优控制、推力幅值与方向的凸约束以及状态与控制的耦合关系。未来可将本文方法推

广至再入制导和运载火箭回收等场景，构建统一的凸优化制导框架。

综上所述，本文在火星动力下降 SOCP 轨迹优化问题上实现了从理论建模到嵌入式验证的完整闭环，并通过开源发布为相关领域的研究者和工程师提供了可复现的技术参考。文中揭示的数值精度现象——包括双精度的必要性、对数变换的误差放大效应以及稀疏矩阵硬件加速的局限性——对于同类优化问题的实现具有普遍借鉴意义。我们希望这项工作能够为航天嵌入式制导算法的工程实践提供一份有价值的参考。

参考文献

- [1] R. D. Braun and R. M. Manning, “Mars exploration entry, descent, and landing challenges,” *Journal of Spacecraft and Rockets*, vol. 44, no. 2, pp. 310–323, 2007.
- [2] A. D. Steltzner, A. Miguel San Martin, T. P. Rivellini, and A. Chen, “Mars science laboratory entry, descent, and landing system development challenges and preliminary flight performance,” in *23rd AIAA/AAS Spaceflight Mechanics Meeting*, pp. 521–528, 2013. Advances in the Astronautical Sciences, Vol. 148, Paper 2013-0236.
- [3] A. Nelessen, C. Sackier, I. Clark, P. Brugarolas, G. Villar, A. Chen, A. Stehura, R. Otero, E. Stilley, D. Way, K. Edquist, S. Mohan, G. Giovengo, and L. Lefland, “Mars 2020 entry, descent, and landing system overview,” in *2019 IEEE Aerospace Conference*, pp. 1–20, 2019.
- [4] P. Ye, Z. Sun, W. Rao, and L. Meng, “Mission overview and key technologies of the first mars probe of China,” *Science China Technological Sciences*, vol. 60, no. 5, pp. 649–657, 2017.
- [5] B. Açıkmese and S. R. Ploen, “Convex programming approach to powered descent guidance for Mars landing,” *Journal of Guidance, Control, and Dynamics*, vol. 30, no. 5, pp. 1353–1366, 2007.
- [6] C. R. Hargraves and S. W. Paris, “Direct trajectory optimization using nonlinear programming and collocation,” *Journal of Guidance, Control, and Dynamics*, vol. 10, no. 4, pp. 338–342, 1987.
- [7] G. N. Elnagar, M. A. Kazemi, and M. Razzaghi, “The pseudospectral legendre method for discretizing optimal control problems,” *IEEE Transactions on Automatic Control*, vol. 40, no. 10, pp. 1793–1796, 1995.
- [8] L. Blackmore, B. Açıkmese, and John M. Carson III, “Lossless convexification of control constraints for a class of nonlinear optimal control problems,” *Systems & Control Letters*, vol. 61, no. 8, pp. 863–870, 2012.
- [9] X. Liu, P. Lu, and B. Pan, “Survey of convex optimization for aerospace applications,” *Astrodynamics*, vol. 1, no. 1, pp. 23–40, 2017.

- [10] L. Blackmore, B. Açıkmeşe, and D. P. Scharf, “Minimum-landing-error powered-descent guidance for Mars landing using convex optimization,” *Journal of Guidance, Control, and Dynamics*, vol. 33, no. 4, pp. 1161–1171, 2010.
- [11] B. Açıkmeşe, John M. Carson III, and L. Blackmore, “Lossless convexification of nonconvex control bound and pointing constraints of the soft landing optimal control problem,” *IEEE Transactions on Control Systems Technology*, vol. 21, no. 6, pp. 2104–2113, 2013.
- [12] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004.
- [13] A. Domahidi, *Methods and Tools for Embedded Optimization and Control*. PhD thesis, ETH Zürich, 2013.
- [14] A. Domahidi, E. Chu, and S. P. Boyd, “ECOS: An SOCP solver for embedded systems,” in *European Control Conference (ECC)*, pp. 3071–3076, 2013.
- [15] T. A. Davis, *Direct Methods for Sparse Linear Systems*, vol. 2 of *Fundamentals of Algorithms*. SIAM, 2006.
- [16] A. Wächter and L. T. Biegler, “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006.
- [17] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, “CasADi: A software framework for nonlinear optimization and optimal control,” *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.
- [18] S. Diamond and S. Boyd, “CVXPY: A Python-embedded modeling language for convex optimization,” *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.
- [19] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, and M. Diehl, “acados—a modular open-source framework for fast embedded optimal control,” *Mathematical Programming Computation*, vol. 14, no. 1, pp. 147–183, 2022.